UCL

Université Catholique de Louvain
Ecole Polytechnique de Louvain

Design of a microsimulation DSL to better estimate adult mortality in countries

Promotors:
Pierre Schaus
Bruno Masquelier
Reader:
Kim Mens

Master's thesis submitted for the graduation of
Master in Computer Science and Engineering
by Dimitri Dufrasne
and Romain Wallez

Louvain-la-Neuve
Academic year 2013 - 2014

Abstract

Microsimulation tools are used since the 70s to provide highly detailed analysis of activities such as demographic evolution, adult mortality, etc. Those programs remain problem specific by answering only to a given problem. Moreover many tools are aged and support may be difficult or no longer available.

To be able to tackle a variety of demographic challenges, the software must be fully parameterizable and should be understandable by a non-programmer. Existing tools contain "holes" or hooks in the program that can be completed only by a programmer in order to achieve a little modularity and extend the program to address a new specific problem.

The aim behind this master thesis is to develop, with help of a modern programming language, an up to date tool for the Univeristé Catholique de Louvain that answers general demographic questions about adult mortality. To achieve this, knowledge and concepts about former tools such as Socsim as well as modern programming techniques will be used.

The new software: MiSca (**M**icrosimulation **i**n **Sca**la) is written in Scala, a concise, expressive and high-level programming language. Its development is based on the Socsim program and enhances this last one by using a domain specific language to help its utilization by demographers. The program is designed to be parameterizable in order to create specific and precise demographic scenarios.

Comparison with other demographic tools such as Socsim yields conclusive results that are very similar to each other. The MiSca program, while equaling the results of Socsim, also provides greater expressiveness and better handling.

Acknowledgements

In this section the authors would like to especially thank individuals and department.

We would like to express our appreciation to Schaus Pierre for the discovery of DSLs during his course of *Languages and Translators* and for the guidance throughout this master thesis.

Masquelier Bruno and the demographic group in the ESPO department at UCL, for providing the Socsim source code and other useful references that helped us to expand our knowledge in the domain of microsimulation.

We would also like to thank Mens Kim who accepted to give time for the reading of this master thesis.

Bogaert Patrick, professor of *Probability and statistics (I)* course, for validating our probability model which is intended to generate randomized events.

All the readers: Deslé Julie, Wallez Jean-Noël, Dufrasne Barbara, Liégeois Cindy, Blaise Benjamin to improve the spelling and writing style.

Contents

1 Introduction **1**
- 1.1 Context . . . 1
- 1.2 Problem . . . 2
- 1.3 Motivation . . . 2
- 1.4 Objectives . . . 3
- 1.5 Approach . . . 3
- 1.6 Contributions . . . 4
- 1.7 Roadmap . . . 4

2 Background material **7**
- 2.1 Microsimulation . . . 7
  - 2.1.1 Description . . . 7
  - 2.1.2 Definition . . . 8
  - 2.1.3 Microsimulation and macrosimulation . . . 9
  - 2.1.4 Interests of microsimulation . . . 9
  - 2.1.5 Limits of microsimulation . . . 10
- 2.2 Domain specific language . . . 11
  - 2.2.1 Definition and description . . . 11
  - 2.2.2 Interests of DSLs . . . 12
  - 2.2.3 Disadvantages of DSLs . . . 12
- 2.3 Overview of several types of simulation . . . 13
- 2.4 Demographic models . . . 14
  - 2.4.1 Lee-Carter model . . . 14
  - 2.4.2 The Brass logit model . . . 15
  - 2.4.3 Coale and Demeny regional model life tables . . . 15

3 Related work **17**
- 3.1 Microsimulation tools . . . 17
  - 3.1.1 Socsim . . . 17
  - 3.1.2 KINSIM . . . 27
  - 3.1.3 CAMSIM . . . 27
  - 3.1.4 LifePaths . . . 27
  - 3.1.5 NetLogo . . . 27

4 Problem statement **29**
- 4.1 Who should use this program ? . . . 29
- 4.2 Technological choices . . . 29

5 Running example **31**
5.1 Development and running environment 31
5.2 How to run a simulation 32

6 Solution **35**
6.1 Goal of MiSca 35
6.2 Description of MiSca 35
6.2.1 Basic principles 36
6.2.2 From Socsim to MiSca 36
6.2.3 Person 37
6.2.4 Events 39
6.2.5 Groups & Transition 40
6.2.6 *onChange* operation 40
6.2.7 Scheduling process 41
6.2.8 Graphical Interface 42
6.3 Core Implementation 44
6.3.1 Core 44
6.3.2 Generating waiting times based on rates 47
6.3.3 Sections 48
6.3.4 Events 50
6.3.5 Person 55
6.3.6 Maps 56
6.3.7 SimuTypes 58
6.4 DSL 65
6.4.1 User Manual 65
6.4.2 Domain-specific semantics 69

7 Validation **79**
7.1 Method 79
7.1.1 Unit Test methods 79
7.1.2 Comparison with Socsim 81
7.2 Results 82
7.2.1 Test block 1: Validating model 82
7.2.2 Test block 2: Validating the new scheduling process . 89
7.3 Analysis 92
7.3.1 Test block 1: Analysis 92
7.3.2 Test block 2: Analysis 93
7.4 Threats to validity 94
7.4.1 Previous model used 94
7.4.2 Pseudo random generation 96
7.4.3 Time complexity 96
7.5 Conclusion from the analysis 98

8 Future work **99**

9 Conclusion **101**

10 Appendix **103**
10.1 Input and Output File of the Scheduling validation 103
10.1.1 Input_file 103
10.1.2 UML events 103
10.1.3 Output_file 100 years 105
10.1.4 Output_file 250 years 106
10.1.5 Transition Rates 107
10.2 Rate Table Test Blocks 109
10.3 Running example & GUIs 109

Chapter 1

Introduction

This document aims to present the design of a microsimulation DSL to better estimate adult mortality in countries.

This first chapter will introduce the context of this master's thesis, followed by a description of the problem tackled. Then the authors will motivate the relevancy of this problem, which will induce the presentation of the objectives and the approach adopted. This chapter will be ended by explaining the contributions of this work and presenting a roadmap of the remainder of this document.

1.1 Context

In some countries, the population is not surveyed properly. Either the information (i.e., ages, dates, kinship, cause and date of death) collected in population censuses is of poor quality or is either unavailable. From this defective data, it is difficult to process a reliable demographic study of this population. Therefore demographers must use other tools to perform these studies.

To address this problem, simulation can be used in this context. There exist different kinds of simulation models for demographic analysis of population. Three of these models are [6] :

Analytical models use mathematical expressions, often continuous.

Microsimulation models apply a sequence of competitive events to individuals of a population.

Macrosimulation models are a mix between analytical models and microsimulation. This type of models does not work at the individual level but simulates demographic processes on subgroups in the population (e.g. individuals with same matrimonial status).

Unlike analytical models, microsimulation and macrosimulation models use a sequence of state transitions. They simulate the dynamic evolution of a population subject to demographic events.

These kinds of demographic tools are therefore very useful to analyze dynamics of population and demographic phenomena.

Despite a large amount of microsimulation models, most of them are often very static in the sense that the interaction with the simulation process is impossible or difficult. Socsim [16] is an example of such a tool performing microsimulation. When executing a simulation with Socsim with some inputs, raw data are obtained as output that demographers have to analyze (with tools like R) at the end of an entire simulation. In this situation, the user does not have much control on the simulation procedure[1] .

1.2 Problem

As explained in the previous section, there are countries for which no civil registration system exists. In such a case, demographers can only rely on incomplete data to perform demographic studies[2]. Obviously these deficient data disrupt analyses. Therefore some tool is needed to extrapolate such information, based on some demographic properties (e.g. mortality rates, etc.). [6]

Moreover, existing tools are quite restricted to a specific problem, such as Socsim. Therefore there is a need for a dynamic, powerful, generic and easy to handle microsimulation tool to analyze demographic phenomena on population (e.g., mortality).

1.3 Motivation

To answer demographic questions, existing tools are created but mainly tackle a specific problem. Other tools try to be less specific and to be able to tackle any general problem, however the latter are not extensible.

Besides the number of existing microsimulation tools, they are aging (Socsim created in the 70s , Camsim in 1987) or are more recent but is not well suited with our microsimulation objectives.

Moreover, tools like Socsim and Camsim are open source and can be enhanced by anyone. Even if support for this aged tools may still be available, it cannot be guaranteed in the next 5 or 10 years. A locally managed tool elaborated by Université Catholique de Louvain will ensure the program's availability for the demographic department (ESPO).

[1] In fact Socsim provides hand-hook to extend it but the user has to be able to develop his enhancement in C language

[2] "For example, survival of parents and siblings collected in census, can be used to estimate adult mortality".[6]

1.4 Objectives

During the development of this work, some objectives were followed. The main one focuses on developing a microsimulation tool with the following properties:

Validity : The tool has to include most of the properties of Socsim. To lead the development of this new tool, we are trying to achieve similar results to those obtained with Socsim. Having this goal will allow to move forward more easily in development.

Easily modifiable & very flexible : The program will be extended with a DSL in order to easily configure simulations. This extension aims to make the tool very flexible.

Communicative : One of the major objectives is to design a DSL. This one should be explicitly communicative to the target audience, i.e. when domain experts are reading some code snippets, they should be able to *understand which business rules the abstraction implements and whether it adequately covers all possible domain scenarios.* [3]

Simplicity : Another important objective is to keep the core of the tool simple. In this way, the core will be more maintainable and easily extensible.

Extensibility : Although one of the goal is simplicity, some features will be provided to enable users to extend easily and efficiently the tool.

Using Scala : One of the recommendations is to use Scala [12]. This is not just a suggestion, Scala has a lot of interesting features that are very helpful in order to achieve the other objectives. Among its numerous qualities, Scala has supports for functional programming and strong static type system, making Scala programs very concise and expressive.

1.5 Approach

In order to solve the problem and to achieve the objectives properly, a clear and simple operating method is required. Here is the one followed:

As one of the recommendation of our model is to be based on Socsim, the first step consists of an analysis and understanding of how Socsim works. This also includes study of the state-of-the-art in the domain of *microsimulation* and *domain specific language*.

The second step is the development of the simulator core based on the idea of discrete event simulation, while bearing in mind to keep it small.

Then comes the development of the resources specific to the domain, i.e. population simulation in our case.

A next step is the development of utilities such as: reader and writer of input and output data files, respectively. Such utilities aim to make MiSca compatible with the same kind of data files used by Socsim. Moreover, in order to be able to easily analyze the coherence of simulation results from MiSca, development of graphic tools proved very helpful.

Finally, such utilities were very useful in order to validate MiSca and to compare it with Socsim. This comparison helps to improve the performance of our solution.

1.6 Contributions

This section aims to highlight major contributions of this master's thesis.

Microsimulation model This master's thesis contributes to design an enhanced microsimulation model based on Socsim. Its major contribution comes from interesting properties developed for MiSca, such as flexibility and extensibility.

DSL This master's thesis also contributes to the exploration of expressiveness of DSLs in the domain of microsimulation. The programming language Scala has capabilities that can be extensively exploited to design an expressive DSL. The latter gives rise to concise script in the DSL language, full of meaning for experts in the domain. This ones read the business rules of the script as easily as a common domain related expert report. In this way, they are able to easily check the consistency of the rules or correct them.

Development of the DSL will be the opportunity to discover the large amount of features offered by Scala. Those components play a major role the DSL's expressiveness. These include advanced static type system allowing to *design purely typed embedded DSLs without resorting to code generation techniques, preprocessors, or macros*[3]. This static type system adds consistency to the DSL abstractions. Another example of the strength of Scala is that it includes functional-oriented features, such as type inference, higher-order functions, currying, pattern matching, etc.

1.7 Roadmap

The remainder of this master's thesis is structured as follows :

Background material This chapter provides the background material necessary for understanding the remainder of this master's thesis. First, microsimulation is defined and discussed, then follows the introduction to domain specific languages (DSL). Finally some demographic models are discussed.

Related work This chapter focuses on a state-of-the-art review in the domain of microsimulation, starting from a historical overview to end with a brief description of some microsimulation models and a detailed presentation of Socsim.

Problem statement This chapter states the problem tackled by this master's thesis. It aims to set the context of this work and to position it with respect to existing related work. And finally it motivates some designing choices (technology : Scala, microsimulation, DSL, ...)

Solution The proposed solution to the initial problem is introduced and discussed in this chapter. First, the goal of MiSca is presented, followed by a conceptual description of this one. Finally implementation details are covered : beginning with the core of the simulator, followed by the DSL design and visual tools presentation.

Validation This chapter presents the validation process conducted in order to validate the solution. Methods used to ensure high-quality programming code are first described. Then come experiments to compare results obtained with MiSca and Socsim. This chapter concludes by an analysis of this validation process results.

Conclusion This chapter aims to analyze and conclude the work carried out during this year. We take this opportunity to check that the objectives are achieved and to deliver the main contributions of this master's thesis.

Future work This chapter suggests a few avenues for improvement.

Chapter 2

Background material

In this chapter, some material will be explained to the reader in order to understand the remainder of this thesis. In this way this paper will be as self-contained as possible.

First, *microsimulation* will be described and defined. This will lead to explore its interests and limits.

Then *domain specific languages* are presented, as well as its interests.

Finally some models related to MiSca will be briefly covered.

2.1 Microsimulation

Initiated in the 1960s, the microsimulation has experienced a significant increase of interest in recent decades, especially through the increase in computer capacity. It is widely used today in many situations : generally to estimate effects of public policies, to analyze spreading disease through a population, etc.[7]

One of the main goals of microsimulation is often to evaluate the effects of proposed interventions before they are implemented in the real world [20].

2.1.1 Description

Bruno Masquelier explains that demographic estimation is based on many imprecisions and bias[6]. These estimations can be improved via use of simulation, allowing to analyze impact of bias for example. Three types of simulation are explained in his thesis:

Analytical models are based on mathematical relations between population sub-groups.

Microsimulation is a model in which persons are subject to competitive events such as *birth*, *death*, *marriage*, *divorce*, etc. In this way, population moves from one state to another.

Macrosimulation is a model based on the transition between several states as the microsimulation. Unlike microsimulation, macrosimulation does not work anymore at the individual level but now at the aggregated level, i.e. demographic processes are simulated on subgroups of persons sharing the same properties.

The micro and macrosimulation enable to make a projection: make propositions on the future of the studied population. As they refer to future, they contain the time variable and are a dynamic model. Indeed, these models of population are dynamic because they show evolutionary process due to events applied to population. These types of simulation are based on a simplified description of real world.[4]

2.1.2 Definition

According to Bruno Masquelier, microsimulation "does not aim to perform new demographic estimations (mortality, fertilization rates, etc.) but works on hypothetical population having complex interactions". In microsimulation, dynamics of the fictional population is split up into series of stochastic events experienced by individuals of the population. These events are defined by individual characteristics such as age, sex, marital status, etc. [6]

The International Microsimulation Association [1] explains that the microsimulation operates at the level of individual units such as persons, vehicles etc. Each unit is characterized by unique identifier and a set of associated features (for example: "a list of vehicles with known origins, destinations and operational characteristics" [20]). These units are subject to competitive risks leading to simulated changes in state and behavior.

There are two approaches for quantitative risk assessment: the deterministic approach and the stochastic approach. The first focuses on a cause that always induces the same effect (probability = 1), such as "changes in tax liability resulting from changes in tax regulations" [20]. The second focuses on the estimation of the probability of occurrence of this risk (probability $\leq$ 1), such as "chance of dying, marrying, giving birth or moving within a given time period"[20]. "In either case the result is an estimate of the outcomes of applying these risks, possibly over many time steps, including both total overall aggregate change and (importantly) the way this change is distributed in the population or location that is being modeled." [20]

If we assume that microsimulation is consistent with real demographic processes, final populations obtained from microsimulation are reliable for concrete analyses. In addition to be globally coherent with aggregated (macro) level models, microsimulation gives more information than those models : final populations provide information about the *variability of the demographic trajectories at the individual level*. [7]

2.1.3 Microsimulation and macrosimulation

As said above, the micro and macrosimulation mimic dynamic process based on underlying events. These events are random variables associated with some probability of occurrence, which can be expressed in an expected value. "This expected value does not have the same status in micro and macrosimulation [...] A macrosimulation model assumes the size of the population is so high that the projected number of events can be considered as equal to its expected value. A microsimulation model supposes that the number of repetition of the random experiment in the sample is so high that the projected number of expected events will be *approximately equal to its expected value*[1]." [4] As it is a probabilistic approach, the projection model has to provide information about the variations around the expected value. The macrosimulation ignores randomness as opposed to microsimulation, in which the randomness is modeled.

Moreover, microsimulation provides a better overall aggregate logic and more information about individual variability. Indeed, microsimulation differs from more conventional methods in operating at the level of individual data rather than aggregated data, and in being based on repeated random events rather than average numbers [4].

Evert Van Imhoff and Wendy Post [4] explain that three components are essential to tell the difference between the two simulations:

- "the model uses a sample of people rather than the entire population"
- "the model works at individual level rather than at group level"
- "the model is based on repetitions of the random experiments rather than average proportions"

They point out that the microsimulation has more possibilities than macrosimulation and is a powerful tool for demographic and no demographic projections. (Evert Van Imhoff and Wendy Post pg 890).

2.1.4 Interests of microsimulation

Several interests of microsimulation can be demonstrated:

The first interest of microsimulation is the randomness: as the microsimulation is based on repetition of the random experiments, two series of microsimulation provide two population results as opposed to macrosimulation based on average data. Bruno Masquelier [6] explains that the randomness of microsimulation is due to competitive risks. Indeed, the occurrence of

[1] Expected value, $\mathbb{E}$, refers to the value of a random variable one would expect to find whether one could repeat the random variable process an infinite number of time and take the average of the values obtained

next event/risk is defined by waiting time shorter. The waiting time for an event is obtained by a translation of the event risk. So the event with a shorter waiting time occurred before events with a longer waiting time.

The second advantage of microsimulation is the updating of variables characterizing individuals when an event occurs.

A third advantage of microsimulation over macrosimulation is that it can trace kinship networks.

The fourth interest of this type of simulation is that interactions between variables are taken into consideration such as "interactions between population process and between people"[6]. It is easier to compare interactions between people such as composition of couples, dissolution of couples, widowhood, etc. In microsimulation, there is an internal coherence that cannot be found in macrosimulation.

Finally, all the dates of occurred events are saved in records for each person. In this way, it is possible to perform classic demographic method on it. Moreover, it is possible to establish the bias due to changes.

The interest of microsimulation in our problem is that this approach enables a direct observation of final population. All necessary statistics are available, even for individual.

2.1.5 Limits of microsimulation

One of the major problems of microsimulation is that it uses a large amount of data, e.g. fertility rates varying according to age, parity, sex and marital status. It results in a huge table of rates. Let's imagine the same kind of table for other rates, it quickly becomes a huge bulk of data that are difficult to handle. As it was already stated in the problem statement, such empirical data does not exist for developing countries and is hard to access for most countries. Therefore, as it is difficult to rely on empirical rates tables, modeling of demographic behaviors are required. Consequently, the conclusions drawn from microsimulations are very dependent on the quality of the model used for demographic behaviors.

Ruggles [14] observes another limit of microsimulation : *"models of kinship assume that most demographic events occurring within kin group are independent of one another. It means that, in most models, the characteristics of one member of group of kin are assumed to be entirely uncorrelated with the characteristics of other members of the kin group"*. He calls that the Whopper Assumption. By contrast, *"in real populations, members of same kin group tend to share many of the same characteristics (i.e. a demographic family resemblance)"*. Indeed, as members of a family typically belong to a same ethnic group, practice the same religion and reside the same region, we observe that they share some demographic behaviors. At most members of the same kin group should resemble one another in demographic behavior

more than they resemble random persons.

Evert Van Imhoff and Wendy Post point out two other limits: Softwares of microsimulation are difficult to use and it is complicated to transfer the computer applications from a model to another.

2.2 Domain specific language

2.2.1 Definition and description

Most programming languages are *general purpose programming languages* (GPL), meaning they are not designed for a specific problem and, therefore, they do not include domain-specific features. This kind of computing language, such as Java, Scala, C, etc., can be used to solve any problem, regardless of the target domain[2].

In contrast, a domain specific language (DSL) is a computer language designed for a target specific domain. *It contains the syntax and semantics that model concepts at the same level of abstraction that the problem domain offers.*[19] [3]

In order to demonstrate the usefulness of DSLs, consider an application whereby some rules have to be expressed in a readable and explicit way. For example one could need to express a time period such as `time() - 1209600`. Providing users with expressive constructs (DSL), such as `2.weeks.ago` for this particular example, can have huge impact on their understanding and efficiency. [3]

We can classify DSLs related to the way they are implemented. Here is a description of the two main types:

Internal DSLs (or embedded DSL) This type of DSL is designed on top of an existing well suited language (e.g. Scala, Ruby, etc.), using its features and infrastructure, to provide a domain-specific semantics. Therefore an internal DSL is bound to the host language syntax and semantics, and uses the host language infrastructure.

External DSLs This type of DSL is based on a language processing tool such as interpreter or compiler to translate the DSL language to the host language. This tool includes lexical analyzers, parsers, code generators. We observe that designing an external DSL is comparable to implementing a new language. Therefore designer of an external DSL is completely free to define its own syntax and semantics.

[2]However, it is worth noting that some languages are better suited to solve specific problem.

2.2.2 Interests of DSLs

Domain Specific Language (DSL) has recently gained popularity mainly due to *progress in programming paradigms and languages. It has made most modern object-oriented, functional programming languages, including Scala, Python and Ruby, expressive enough to define embedded DSLs* [9]

Scala supports most of the *features of functional languages*, including functions as first class values, closures, list comprehensions, curried functions, lazy evaluation and pattern matching. Such features make Scala well suited to design DSLs [9]

High level of abstraction A DSL provides a higher level of abstraction. The user only has to focus on the problem, not on the implementation details.

Correct level of expressiveness Being specific to a domain, a DSL has to be communicative to the domain users. This means that a DSL script must be understandable to non-programming users expert in the domain. To this end, the DSL has to provide appropriate syntactic and semantic abstraction level. That is, the DSL must offer a *explicitly communicative API*[3].

Conciseness This property involves that DSLs are easy to manipulate and think about.

Expressiveness DSLs contain abstractions that speak the precise semantics of a domain.

Higher payoff As DSLs only focus on the domain problem modeling, programming with a DSL boosts efficiency.

2.2.3 Disadvantages of DSLs

However DSLs have some minor disadvantages as well. Here are some of them:

Limited expressivity While a general purpose programming language can model everything, a DSL is specific to a domain and can, therefore, only solves problems related to this domain.

Performance concerns As DSLs add a layer in the implementation infrastructure, DSLs can cause performance concerns.

[3]Application Programming Interface (API): a set of tools (class, methods, etc.) that are provided by the author in order to specify how software components should interact with each other.

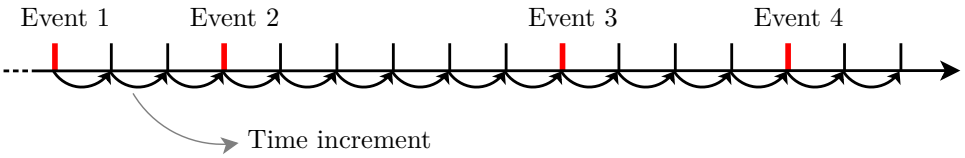


Figure 2.1: Activity based simulation

2.3 Overview of several types of simulation

"Discrete Event Simulation concerns the modeling over time of system with a chronological sequence of events. Each event occurs at an instant in time and marks a change of state in the system." [15]

Continuous simulation is distinguished from discrete event simulation in that it "concerns the modeling over time of a system by a representation in which state variables change continuously with respect to time." [15]

Here we present briefly an overview of the three major discrete event simulation paradigms [8]:

Real Time Simulation This type of system simulates at real time speed. The main drawback of such paradigm is clear: the simulation requires to last as long as the reality time; e.g. imagine one that simulates the life of a person, he probably does not want to wait so much time to get the simulation results.

Activity Based Simulation The time is broken into tiny increments. At each interval, every objects of the simulation are explored to check if an event is happening in the time slice. The state of the system is updated accordingly. The Figure 2.1 shows the drawback of this kind of simulation: there is a check of the existence of event for each object and this check is processed at each fixed time increment. This idea of fixed time increment causes an important waste of computing resources. It is very slow to execute because most time increments will involve no change of the system state.

Event Based Simulation Instead of moving forward in time by fixed time increment (as modeled in the activity based simulation), simulated time is advanced to the time of the next event (as shown in the Figure 2.2). In this paradigm, an object is called only when it has an event scheduled at the current time. There are, therefore, jumps in time between scheduled events. Computing resources are used only we it is need, i.e. for events execution.

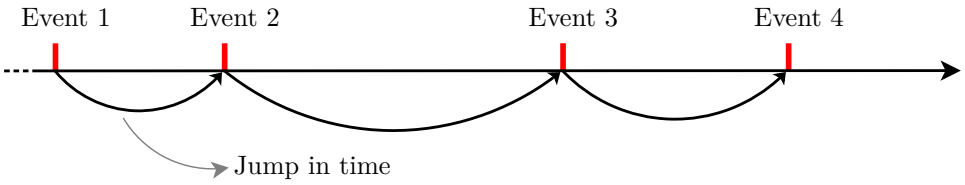


Figure 2.2: Event based simulation

2.4 Demographic models

2.4.1 Lee-Carter model

The Lee-Carter model (JASA article 1992) is a numerical algorithm that is used in life expectancy and mortality forecasting. The input to the model is a matrix that is ordered monotonically by time that contains *mortality rates* associated with ages and years. The output is another forecast matrix of mortality rates. The model uses the *singular value decomposition*[4] to find an *univariate time series vector* k_t [5] that captures 80 to 90% of the mortality trend. "A distinctive feature of this is the usage of stochastic process to model uncertainty about the future"[13]. To be able to do projections, Lee and Carter adjust the logarithm of mortality rates by age using the following expression:

$$log(m_{xt}) = a_x + b_x * K(T) + \epsilon_{x,t}$$

where *log* is the natural logarithm. a is a vector that can be interpreted as an average age profile and b a vector that determines how much each age group changes when $K(T)$ changes. The $K(T)$ parameter tracks the evolution of the intensity of the mortality over time t. Lee and Carter estimated the a, b and K with the U.S. mortality data from 1933 to 1987 using least squares reduction. They estimate a averaging log-rates over time and b and K via a singular value decomposition of the residuals. The k's estimation is performed next in order to predict the correct total number of deaths each year. The results show that the $K(T)$ parameter has decreased linearly during the twentieth century[13].

A second distinctive feature of this approach is that, having reduced the time dimension of mortality to a single index $K(T)$, Lee and Carter use statistical time series methods to model and forecast this index. During their application to the U.S. mortality, they reveal that index behaves like

[4]Wikipedia : In linear algebra, the singular value decomposition (SVD) is a factorization of a real or complex matrix, with is useful in statistical applications.

[5]The term univariate refers to an expression, function or polynomial of only one variable. The term is commonly used in statistics in order to distinguish a distribution of one variable from several ones.

a simple random walk with drift :

$$K(T) = K(T-1) + d + \epsilon_t$$

where d is the drift currently set as -0.365 and the ϵ_t are independent error terms with variance v, estimated to 0.652^2. This model is currently used in Socsim, the three parameters (a_x, b_x, $K(T)$) can be provided or set during the mortality rate table creation. We will use the same model in order to validate our results with Socsim.

2.4.2 The Brass logit model

Another model that gives greater flexibility is the *logit* system. This model relies on two different life tables. "Brass discovered that a certain transformation of the probabilities of survival to age x made the linear relationship between corresponding probabilities for different life tables approximately linerar"[11]. The linear relationship for constructing life table system is given in the next expression:

$$\lambda(l*(x)) = \alpha + \beta\lambda(lx))$$

where $l*$ and l are two different life tables and α and β are constants that can be determined such that for all age x between 1 and σ. "If the above equation holds for every pair of life tables, then any life table can be generated from a single standard life table by changing the pairs of (α, β) values used[10]".

$$\lambda(l(x)) = logit(1.0 - l(x)) = 0.5\ log((1.0 - l(x))/l(x))^6$$

2.4.3 Coale and Demeny regional model life tables

Published in 1966, those regional life tables are derived from a set of 192 life tables. Those tables reveals that four mortality patterns can be identified. Those one are *North*, *West*, *South* and *East* that correspond respectively to regions around the globe. Those tables can be found in the Manuel X[11], a short resume of what they are composed is undermentioned:

- North model - comes from Iceland (1941 - 1950), Northway (1856 - 1880 and 1946 - 1955) ans Sweden (1851 - 1890). This table is characterized by a low infant mortality coupled with relatively hight child mortality. Adult mortality at age 50 falls increasingly.
- East model - originated from Austria, Germany, central Italy and Poland. The life expectancy of those tables goes from 36.6 to a high of 72.3 years.

[6]The logit is defined in statistic with a probability p rather than a λ which is used with the form: $logit(p) = 0.5log(p/(1.0-p))$, where the *log* is the natural logarithm.

- South model - Spain, Portugal, Italy (1876 - 1957). With a life expectancy of 35.7 years to 68.8 years. Characterized by a high mortality under age 5, a low one about 40 to 60 and high mortality rate over age 65.
- West model - This model is used and recommended for representing mortality in countries where there is a lack of evidence. Life expectancy goes from 38.6 years (Taïwan, 1921) to 75.2 years (Sweden, 1959).

This model is used to generate life tables that are used in Socsim and moreover the West model serves as input rates in our validation section.

Chapter 3

Related work

This chapter will introduce some existing work about microsimulation.

First, five major tools used to tackle the master's thesis problem are presented.

Then, a look at the history and the state of the art of microsimulation are addressed.

Then comes a short discussion about some tools used in microsimulation. Socsim is presented in more detail.

3.1 Microsimulation tools

In this section we present some existing microsimulation tools.

3.1.1 Socsim

Socsim was created in the early 70's by Peter Laslett, Eugene Hammel and Kenneth Wachter at the Univeristy of California, Berkeley. Socsim adds the concept of households to the microsimulation model; therefore it was used to model the composition of preindustrial household villages in England over a period of 150 years. Various assumptions for demographic rates and living conditions were applied during the simulation (Hammel et al., 1976). Other application of Socsim were identified in the United-States in 2000 to model parental structures (Hammel et al., 1981; Reeves, 1987). Socsim, since its creation, has been constantly updated or modified to handle a specific problem and various implementations still exist nowadays.

Socsim in a few words

The simulation is *activity-based* meaning that events of fictitious persons are executed on a monthly basis. Events are generated according to rate tables, the rates of these ones are translated into a *waiting time*. The occurrence of the next event is defined by the shortest waiting time. During all the simulation each person has exactly one scheduled event, those ones are executed sequentially to comply with their time scheduling. Socsim is currently used (2014) at the Univeristé Catholique de Louvain by the demography department, they are using this tool to generate and survey populations. Socsim

was used as the basis of this master's thesis therefore many concepts and ideas are related to it and must be explained.

Socsim model

The four following points present how Socsim models the various components of the population dynamics

Fertility The fertility may vary per group, age, marital status and parity of a person. Socsim deals with five different matrimonial status: single, married, divorced, cohabitant and widowed. The modeling takes into account fertility rates rather than fertility biological life cycles and/or sexual activities.

Mortality The mortality varies by age, group, sex and matrimonial status. As previously seen in the Section 2.4, Socsim implements the evolution of the mortality based on the parameters of the Lee and Carter's model. Those three parameters of this model are directly introduced into the program. They are used to recalculate the rates.

Nuptiality The marriages in Socsim regroup two sub-events, the first one concerns about the cohabitation between two individuals and the second deals with the marriage itself. When a person wants to enter in a relationship a sort of *pretender pool* is calculated for that person. Success for finding a spouse depends on a scoring function. The result of that union is either a cohabitation or a marriage (this choice is based on the spouses' characteristics).

Groups transitions Socsim handles sub-groups where people can be classified in. Membership of this group is acquired by birth and transmitted by its parents (choice of inheritance must be parametrized by the user) but may vary during the person's life. Group transmission to a child can be: from the mother, the father, and the opposite or the same sex parent. At each time people are subject to group changes, those transitions are based on a rates table set by the user at the beginning of the simulation. If no rates are specified for a group, people will not move in and out of this last one.

Socsim core

This section shows the minimal basis of the Socsim program, more information is detailed in the Socsim oversimplified article [5].

Segments and events Socsim is based on segments, each have a duration of N months. For example a simulation can consist of two segments of a duration of 1200 and 600 months respectively. The purpose of having such

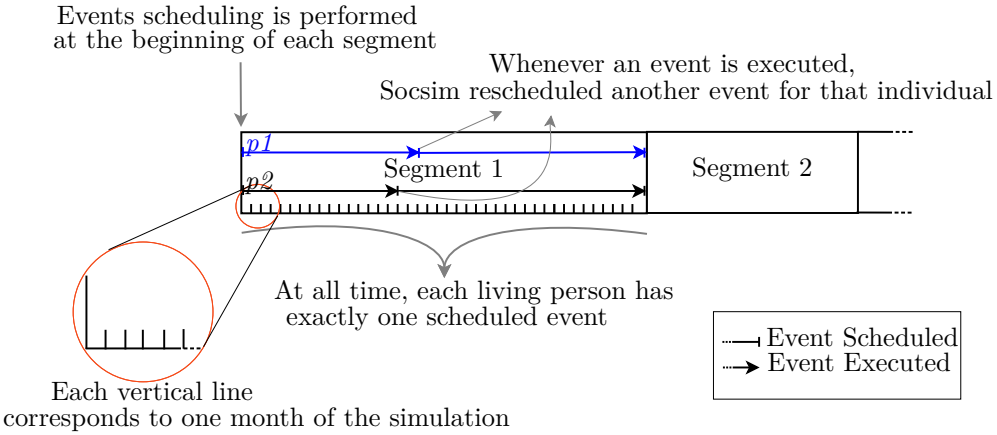


Figure 3.1: Socsim's core

separation segments is to apply to each of them specific rates and other properties such as the birth interval, fertility multipliers, etc.

At all time during the simulation, people still alive have exactly one scheduled event. There are five possible events: birth, death, marriage, divorce and transition from one group to another. Therefore some actions can affect and trigger other events, e.g., the death of a husband will trigger a new event for the wife to change her matrimonial status from married to widowed.

In order to determine which event is destined to which person, Socsim generates a random waiting time associated to the five events. To maintain the socsim's rule: "exactly one scheduled event at each time", the earliest triggered event is chosen and scheduled, the remaining events are discarded.

The figure 3.1 defines the Socsim basics, firstly each segment is defined by its duration, specific rates (if any) and other optional arguments. Every living person must have exactly one scheduled event therefore Socsim attributes at the beginning of each segment one event per person. Once each person has a planned event, the simulation starts. Each event corresponding to the first month is executed in a random order. When all events for this month are executed, Socsim increments the month and moves on to the next one. If there are several events scheduled in this month, they will be executed. This incrementation is done until there are no more events in the list. At the end Socsim writes out output files (.opop, .omar, .opopx)

The figure 3.1 displays how Socsim segments and events are performed, for simplicity this simulation models only two people. The segment 1 has planned the first month with two events: one for $p1$ and another for $p2$. The program executes first the $p2$'s event and immediately re-plans another event for this individual. The event of $p1$ is then executed. As mentioned

above, event of one person can trigger other event. The figure 3.2 shows how event of $p1$ can trigger an event of $p2$ canceling his old planned event.

In the first sub-picture (a), at the beginning of the segment 1, each person has a planned event. $p2$'s event will be the first to be executed. Let say that $p2$'s event is its own death. When this event will be executed (b), it will trigger an event for his spouse ($p1$) in order to change her matrimonial status. As Socsim rules says: there is only one planned event at all time in the simulation, the old upcoming $p1$ birth event will be deleted as shown in sub-figure (c).

Socsim is an activity based simulation meaning that instead of marching through time event by event, time progresses month by month. E.g., if the first event occurs in month 20 and the next event occurs in month 40, the simulation will go from 20 to 40 by increasing month by month the time and checking whether there are events to execute or not.

Waiting times Socsim plans events to a certain month in the future meaning that events must wait a certain time before getting triggered. This waiting time algorithm is equivalent to drawing a random number u, from a uniform $(0, 1)$ distribution. The letter u represents the probability that the event has not been triggered yet. The probability of non-occurrence of an event is given by the formula in figure 3.3

$$\prod_{t=0}^{n} (1 - p_t)$$

Figure 3.3: Waiting time formula

Where p_t is the probability of the event's occurrence on a period t and $(1 - p_t)$ represent the non-occurrence. The formula regroups the product of the non-occurrence until a success. What Socsim does is mathematically equivalent. However the implementation takes advantages that the probabilities can be the same over month, years and works with power of $(1 - p_t)$.

Combination with R

Socsim has chosen to read and write number based files. Those files are multiply linked list, therefore the usage of spreadsheets is profoundly suboptimal. In order to retrieve relevant information about the files, an external software must be used to reorder or transform those data. Programming language or scientific tool (such as Octave, Matlab or R) can be used to parse the files or can be useful for plotting graphs.

The Listing 3.1 shows how socsim's population data file can be manipulated with R to create a gender pie chart distribution.

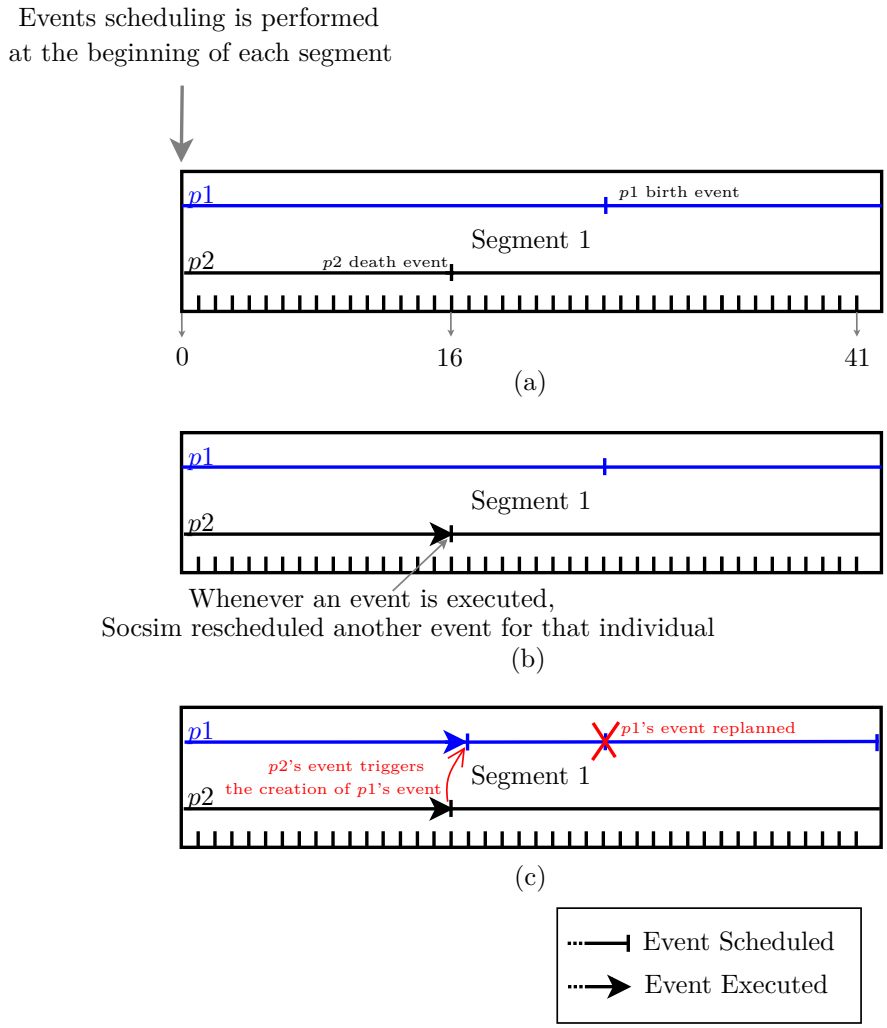


Figure 3.2: Socsim's events competition

```
1  # R code to generate pie chart from the raw data of .opop
   socsim's file
2  P <- read.table(file=paste(filestem,".opop",sep=""),header=F
   ,sep="",
3                    col.names=
4                    c("id","sex","grp","dnv","birth","momid",
5                      "dadid","esibm","esibd","lborn",
6                      "mar","mstat","death","fmult"))
7  # Re-order population list based on ids'
8  P<-P[order(P$id),]
9
10   # Gender distribution
11 Male <- P[P$sex == 0,]
12 Female <- P[P$sex == 1,]
13
14
15 # Create Gender Pie Chart data & label
16 data <- c(length(Male$id),length(Female$id))
17 pieLabels <- round(data/sum(data) * 100 , 1)
18 pieLabels <- paste(pieLabels, "%", sep="")
19 colors <- c("white",  cm.colors(2))
20 # Plot pie chart
21 pie(data, labels= pieLabels, main="Gender Pie Chart", col=
   colors)
22 legend(1.5, 0.5, c("Male", "Female"), cex=0.8,
23  fill=colors)
```

Listing 3.1: Simple R to generate Alive and Gender pie charts

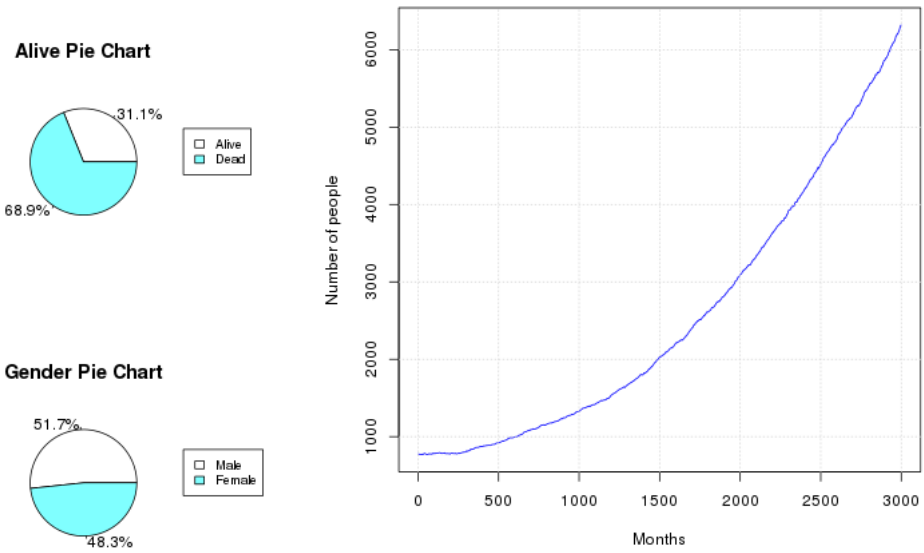


Figure 3.4: Simple R to generate Gender pie chart

Run a Socsim simulation

This section exposes a brief explanation of mandatory input that must be provided in order to run a simulation.
Socsim must be started with two inputs arguments, the first one is a supervisory file **.sup** which regroups the global parameters of the program such as:

- Number of simulation segments N
- Input file name
- Output file name
- Duration N (where N is the number of months that the current simulation must run)
- A "run" directive

Those are the minimal global directives needed for launching a simulation. You can add more properties about the simulation and/or rate files with the "include <path to file> " command. In Socsim, a *segment* is a period of time under which a given set of vital rates and parameters are effective. The "run" command must be provided in order to separate and execute a segment. If the directives for a segment does not end with a run directive, the segment will not be launched. Segment will be detailed in the

following Socsim core section.

The input and output files names must be provided without a file extension. Therefore if your file name is "test_simulation" Socsim will automatically searchs for all the .opop, .omar,... corresponding to that file name. Those extensions regroup populations, marriages and transitions informations.

Generating randomness The second Socsim argument is a random number that is used to generate other pseudo random numbers that are associated with the occurrence of the events. Randomness in Socsim can be provided either with a *seed* number, or a file containing a sequence of random numbers. The first solution is to inject the seed number into the formula:

$$((((Multiplier * seed) + Increment)\%Modulus)/Modulus)$$

where the Multiplier, Increment and Modulus are constants that are set in Socsim. The benefit of this *pseudo randomness* is to be able to generate a sequence of numbers that is generated based on a single number. This can be useful when analyses about the variation of simulation's parameters must be made. In this way, one ensures that the small parameter variations that you introduce in the simulation are the actual changing results and not the results caused by the randomness. In other words, the use of pseudo-random numbers as opposed to true random numbers is a benefit when a simulation needs a rerun with exactly the same behavior. The other randomness technique is to provide a file with a suite of random numbers that will be read from socsim and used it directly.

Socsim files Socsim reads and writes everything it knows about people in two files: the population **.opop** file and the marriage **.omar** file. Both are space delimited files and both contain only numbers. Socsim, when executed, will scan the current folder to find those files. However if the files are not found it will not start. Initial files must be created and all values can be set to 0.

Population & Marriage file The Tables 3.1 and 3.2 regroup respectively the file structure of **.opop** and **.omar** files. As mentioned before those files are number based, as shown in table fields like "mom", "pop", ... Spreadsheet program cannot directly handle such table. Nevertheless some scientific tools like R exist to manipulate those data as shown previously in the Section 3.1.1.

position	name	description
1	pid	Person id unique identifier assigned as integer in birth order
2	fem	1 if female 0 if male
3	group	Group identifier 1..60 current group membership of individual
4	nev	Next scheduled event
5	dob	Date of birth integer month number
6	mom	Person id of mother
7	pop	Person id of father
8	nesibm	Person id of next eldest sibling through mother
9	nesibp	Person id of next eldest sibling through father
10	lborn	Person id of last born child
11	marid	Id of marriage in .omar file
12	mstat	Marital status at end of simulation integer 1=single; 2=divorced; 3=widowed; 4=married
13	dod	Date of death or 0 if alive at end of simulation
14	fmult	Fertility multiplier

Table 3.1: contents and format of the .opop file [5]

position	name	description
1	mid	Marriage id number (unique sequential integer)
2	wpid	Wife's person id
3	hpid	Husband's person id
4	dstart	Date marriage began
5	dend	Date marriage ended or zero if still in force at end of simulation
6	rend	Reason marriage ended 2 = divorce; 3 = death of one partner
7	wprior	Marriage id of wife's next most recent prior marriage
8	hprior	Marriage id of husband's next most recent prior marriage

Table 3.2: contents and format of the .omar file [5]

position	name	description
1	pid	Person id to who the transition event occurred
2	date	Month in which the transtion occurred
3	fromg	group from which the person transitions
4	tog	group into which the person transitions
5	sequence	a non positive number indicating the order of the event. A zero indicates that the current record refers to the most recent transition event; a -7 indicates that seven transitions have occurred to this person subsequent to that of the current record.

Table 3.3: contents and format of the .otx file [5]

Transition history file If the simulation includes group transitions, Socsim will write a file with the same path as the population and marriage files but with the suffix **.otx**. Socsim can output three kinds of files : opop, omar which are similar to the input files in the program input's and a otx transition history file. The Tables 3.1 , 3.2 and 3.3 and their corresponding extension files are still used in the new software in order to ensure a maximal compatibility and R support.

Socsim's Pros & Cons

Pros	Cons
C language very fast...	...but difficult to understand for non programmer who is willing to enhance the socsim code.
.opop, .omar,... data files are program independent for creating charts...	but requires strong knowledge of scientific or programming tools for manipulates raw data.
Socsim planning operation allows only on event at the time for a person (the most recent one)...	Which can lead to event starvation (see Section 7.3)
Socsim allows to enhance the code through hook points...	...but user need to redefine all the multipliers function (fmult, tmult, etc.) (see Section 6.4.2)

3.1.2 KINSIM

Incorportaing the main concepts of microsimulation; KINSIM project, developped by the Netherlands Interdisciplinary Demographic Institute (NIDI), provides an overview of the possible changes in the size and structure of kinship network. Especially for estimating the various possibilities of the assistance for the elderly in an aging population. This simulation tool try to ask questions about demographic aging s.a. "What will be the average number of alive children for elderly people aged 80+ in 2030".[4]

3.1.3 CAMSIM

The CAMbridge SIMulation model was designed in 1987 by James Smith. The purpose of this development was to simulate sets of related and parental grouping during their various steps of their lives. The elaboration of CAMSIM is closely related to SOCSIM as there was a colaboration research between Laslett (SOCSIM developer) and Smith for implementing the SOCSIM program over the Cambridge's computers[22]. The first objective was to have set up SOCSIM on both sides of the Atlantic but due to incompatibility of the computer system and other technical difficulties, SOCSIM was not suited for the Cambridge's computers and a new version was born and named CAMSIM.

3.1.4 LifePaths

LifePaths is a *"microsimulation model designed to simulate life histories: taking account of birth, death, immigration status, inter-provincial migration, marital history (including common-law unions), educational history, employment history and the birth and presence of children at home. It is used to analyze government policies having an essentially longitudinal component and whose nature requires evaluation at the individual or family level, such as post-secondary education costs and benefits or public pension sustainability. It can also be used to explore a variety of societal issues of a longitudinal nature such as intergenerational equity or time allocation over entire lifetimes."*[17]

This model is implemented in Modgen, a microsimulation programming language that support models creation. Modgen was created to automate as much as possible many aspects of the microsimulation model. This program relies on Monte Carlo variation[1].

3.1.5 NetLogo

Netlogo is defined as a *"programmable modeling environment for simulating natural and social phenomena. It was authored by Uri Wilensky in 1999 and*

[1] `http://en.wikipedia.org/wiki/Monte_Carlo_integration`

has been in continuous development ever since at the Center for Connected Learning and Computer-Based Modeling."[21]

Netlogo is an opensource Java oriented software. It relies on a graphical interface where user can create and simulate agents. The simulation is composed of mobile agent called "Turtles" that moves over a grid of stationary agents ("Patches"). Link agents connect turtles to make networks, graphs and aggregates.[21]

Chapter 4

Problem statement

The aim of this master's thesis is to develop a new tool, in parallel with a related domain specific language, to better estimate mortality. Based on previously successful programs such as Socsim discussed in the Section 3. The new tool should be able to reproduce results that are similar to Socsim, while integrating novel features. A major challenge is to provide users, which is familiar with the demographic topic, with an intuitive manner of programming its own simulation scenarios. Previous researches about microsimulation tend to focus on one specific problematic; e.g. Socsim was developed to illustrate the house-holding in England but was updated few years later to handle another specific problem. Another case: KINSIM was developed only to answer about demographic aging.

4.1 Who should use this program ?

As previously mentioned, the demographic department of UCL aims to use this tool but it can be delivered for all other users who want to simulate populations by interacting peoples together through events and collect statistical data.

4.2 Technological choices

DES Demographic microsimulation is based on population simulated at the individual level, where competitive events are experienced by individuals during their life. Since modeling a people's life is practically impossible, the simulation focuses only on relevant events. Those important moments in a person's life can therefore be jumped from one event to another. Based on the discussion of the different type of simulation in Section 2.3, the most appropriate type of discrete simulation is the *Event Based Simulation*. To argue with the two others, the first real-time discrete simulation requires to have a simulation as long as the whole population's life which is impossible to handle. The second one based on activity imposes to check the existence of events in specific time slices which cause unnecessary computer processing.

Scala Modern language tends to offer more scalability, object-oriented languages has been immensely successful since the 60s with Simula and

Smalltalk in the 70s. Scala is a pure object-oriented which is java compatible. All the compiled code runs over a JVM[1] which can be executed on various platforms (x32, x64 bits processors) and various operating systems that support virtual machines (Windows, Mac OSX, Linux, etc.). Scala also contains the functional paradigm which offers interesting features (higher order functions, type inference, pattern matching, etc.). Scala is concise, programs tend to be as short as possible. Scala is high-level, complexity increases with the size of the program and users must be able to understand the code that they are working with. High-level coding helps the programmer and the user to manage the complexity by raising the level of abstraction. In our tool the abstraction mixed with DSL provide with a concise view of the simulation tools for users, while hiding the technical details of the implementation. [12]

DSL Object oriented programming delivers flexible and elegant way of programming but is quite restricted to users who do not have a certain knowledge about programming. Domain Specific Language offers a computer language which is specialized to a particular application domain. The objective here is to create a kind of microsimulation language that can be used by demographers.

[1] Java Virtual Machine: `http://docs.oracle.com/javase/specs/jvms/se7/html/`

Chapter 5

Running example

This chapter aims to introduce the tool developed to deal with the initial problem. Through a running example, all the details about running a simulation with the new tool MiSca will be provided. This will help the reader to have an insight into the use of MiSca.

First, the environment used for the development and for running simulations will be presented. Then an explanation of the configuration and an example of a simulation will be performed.

5.1 Development and running environment

All the project was developed in the Scala IDE. This development environment provides advanced support for development of Scala application. This tool will be used to present the running of a MiSca simulation.

In order to run MiSca, first user needs to install the Scala IDE[1]. Then he needs to get source code of the project. MiSca is available on the following Bitbucket repository : `https://bitbucket.org/romainw/memoire`. All the source code and documentation about MiSca are available at this address.

Once you have imported all the project in the Scala IDE, you some libraries are needed to be able to compile and run the project. Here are the libraries and the Scala and Java versions used to develop MiSca:

Scala Library 2.10.2 Scala edition.

JavaSE-1.6 Java edition.

JCommon 1.0.21 is a *Java class library used by JFreeChart*[2].

JFreeChart 1.0.14 is a *Java chart library to create charts in applications*[3].

ScalaTest 2.1.0 This test framework provides support for writing *clear and simple tests and specifications*[4].

Appendix 10.2 shows the MiSca project opened in the Scala IDE environment.

[1] Scala IDE can be downloaded from `http://scala-ide.org/`

[2] JCommon is available at `http://www.jfree.org/jcommon/`

[3] JFreeChart is available at `http://www.jfree.org/jfreechart/`

[4] ScalaTest is available at `http://www.scalatest.org/`

5.2 How to run a simulation

Once the installation process is complete, you are able to run a simulation with MiSca.

But before looking at the running example, a brief overview of the composition of MiSca will be observed. It can be decomposed into three major modules:

A core implementation containing all necessary pieces to run a simulation. This includes the simulation loop and all the resources such as person representation.

A dsl layer standing above this core. This aims to provide the end user with a simple and expressive language to configure its simulation. It also enables the user to define scenarios of simulation.

Simulation scenarios are defined by users when using dsl. It enables to easily configure a simulation in a communicative way. This means that the written scenarios are fully comprehensive. Experts in the domain are able to understand and verify consistency of the rules expressed in the scenarios, even if they have no programming knowledge.

The running example is described in the following section. Notice that this example will not be used to explore implementation details. This section only provide an insight into running a MiSca simulation. The Chapter 6 will address these details.

The example is a scenario about HIV infection. The rules defining the HIV spreading are listed in Listing 5.1. It should be notice how expressive it is.

In the MiSca project, there is a package named *scenario* which is intended to contain scenarios. For example, the Figure 10.2 shows our scenario *HIVscenario* in this package.

Here is a step by step dissection of the rules defining the scenario in Listing 5.1. The first line defines the package where stands the file. Then come some lines starting with the key word `import` that import some material used in the scenario. And finally, following this lines comes the *object* containing the scenario. In addition to contain the scenario, it names it. For example, if you want to design a disaster scenario, you could probably have a scenario declaration such as :

```
object DisasterScenario extends App
```

It is important to not forget to follow the name of the scenario by `extends App`. This makes your scenario runnable.

The scenario itself defines the following rules:

- Two groups are first defined: one group named *HIV-negative* consisting of uninfected people and one group named *HIV-positive* with people infected by HIV.
- Then the population is loaded from an *.opop* file, the marriages from an *.omar* file and the rates from a rate file.
- Following this data loading, a new transition multiplier is defined.
- Then come two rules definitions:
  - The first one defines the behavior to adopt when getting married. It says that when a person gets married, he/she has to check if he/she or her/his spouse is *HIV-postive*. If so, the transfer multiplier of the spouse and of the person itself become HIVTransMult. It means that the chance to go into the group HIV-positive (i.e. being infected) changes. Notice that the rule

    `tmult of person.spouse is HIVTransMult`

    is equivalent to the more programming one

    `person.spouse.tmult = HIVTransMult`
  - The second one defines the behavior to adopt when entering the Group *HIV-positive*, i.e. when being infected. It says that when a person transits to the group *HIV-positive*, (s)he infects his/her spouses.
- Then come the instruction launching a 250 years simulation.
- And finally the resulting population is written in an *.out.opop* file and all the marriages in an *.out.omar* file.

All that remains to do is to run this scenario. For that purpose, this top-level object *HIVscenario* just needs to be run.

This example aimed to introduce the reader with MiSca in an interactive way. This section was also intended to give the reader all the necessary material to run a simulation with MiSca. All the details are left in the Section 6.

```
1  package scenarios
2
3  import resources._
4  import resources.Duration._
5  import resources.TypeImplicits._
6  import simulator._
7  import simulator.Simulation._
8  import io._
9  import io.Io._
10 import dsl._
11 import dsl.PersonImplicits._
12 import dsl.MultBuilders._
13
14 object HIVscenario extends App {
15
16   Group("HIV-negative")
17
18   Group("HIV-positive")
19
20   loadPopulation("HIVscenario.opop")
21   loadMarriages("HIVscenario.opop")
22   loadRates("HIVscenario.rate")
23
24   val HIVTransMult = 0.5
25
26   Person onMarriage { person =>
27
28     if((person.spouse is "HIV-positive") || (person is "HIV-
       positive")) {
29       tmult of person.spouse is HIVTransMult
30       tmult of person is HIVTransMult
31     }
32   }
33
34   Group("HIV-positive") onTransition{ person =>
35
36     person infects tmult to person.spouses
37
38     Simulation deleteEventsOf person.spouse
39     Simulation randomEventFor person.spouse
40   }
41
42   Simulation simulate 250.years
43
44   savePopulation("HIVscenario.out.opop")
45   saveMarriages("HIVscenario.out.omar")
46
47 }
```

Listing 5.1: Running example: HIV scenario

Chapter 6

Solution

This chapter describes our solution addressed to the initial problem. The goal of MiSca is discussed first. Then comes the description of the simulation core, as well as the presentation of the DSL, followed by a user manual of MiSca. And finally the implementation details will be discussed.

6.1 Goal of MiSca

Most tools only give raw output data at the end of the simulation, this restriction imposes that you cannot interact dynamically with these data during the simulation. MiSca tries to solve this problem by providing tools such as *controlled attributes* or *hooks* to define easily user's scenarios. For example MiSca enables users to trigger some events at a specific time, change rates of one group after several years, etc,.

In order to develop a flexible program, it appears that its core must be as simple as possible. Specific or precise applications related to the user's domain are left to the responsibility of this one. One of the MiSca's tasks is to provide a powerful, user-friendly and simple language which is easier to learn from non-programmer. Domain Specific Language (DSL) are used for this precise matter, the idea is to develop a language based on keywords and sentences of a precise domain, in our context the microsimulation. Therefore people who are familiar with the specific domain can analyze and understand the content of the program without a programming background.

6.2 Description of MiSca

MiSca is a dynamical microsimulation program in *discrete* time. The model is *close*, meaning that people are only created by birth experienced by an existing person and can interact only with people inside the simulation (e.g., marriage only between people of the studied population). In other terms, besides the initial created population, no extra individuals except from births are added during the simulation. The simulation moves forward with events associated to each person. Each living individual is simulated during its whole life and events are generated for him.

6.2.1 Basic principles

At each time of the simulation each alive person has a planned event - During the simulation, each non-dead individual must have a planned event. This principle comes directly from the Socsim principles, it ensures that the person will not remain inactive (alive person with no events) during the whole simulation.

A living person may have one or more planned events - In order to respect the first principle each executed event must launch a planning operation to set up one or more new events. Socsim used to deal with only one planned event at each time by segment and split up time into segment. This limitation leads to trouble when segments are very long. For example if the segment lasts for 1200 months a person who has planned only its own death at the age eighty will remain dormant for the rest of its life[1], although other events such as marriage, or birth would have very high probability to happen at middle age. To remedy this situation, time is no longer split up into segments but sections and the scheduling done at the beginning of the simulation can plan one or more events. Those events may be dropped at the time of execution whether they are not applicable anymore. This ensures that each individual will remain active during the simulation. Moreover this modeling appears to be closer to the real behavior than Socsim model.

A person can be part of one group only - This limitation comes directly from Socsim. The simulation may have as many groups as it needs but a person can be in at most one group at a time. Multi-groups were tested and discussed but rejected (see Section 6.2.5).

6.2.2 From Socsim to MiSca

Based on the previous principles, some ideas from Socsim were kept and other removed.

The **Scheduling process**, Socsim being an *activity based simulation*: an array with the same length of a segment is created at the beginning. Each cell in the array represents a month in the simulation. An activity based simulation will go through month by month checking whether there are events to execute or not. This model raises too much computational time for checking the presence of events in the current month. In the new implementation the simulation is event based with a priority queue, meaning that each event is stored into a queue. This event queue pops event by event and performs a time jump between events. There is no need to move with a monthly basis and many checks operations are therefore avoided.

[1] Unless other individuals interact with this person through events.

The **Segment**, as previously explained in the Socsim section, is created to represent a certain portion of time in the simulation. Rates and specific parameters can be applied to this portion of time. Since long segment introduces dormant persons, MiSca does not plan one event at a time but multiple ones, this ensure that no *rate starvation* will occur during the simulation. To illustrate this starvation let us use an example where the probability of giving birth is very high from 15 to 35 years old and marriage's rates close to the birth's rates but slightly lower. In this example, Socsim will choose mostly the closest event planned which will be the birth. Marriages may occur but are exposed to an unfair competition since it must be firstly triggered and secondly planned closer than the birth event in order to be selected. MiSca does not use events competition that prioritizes them according to the scheduling time. Each type of events may be planned with different times. Validation (such as checking whether a person is married or not) will be controlled when the events will be executed. In this scenario with the same rates as the previous example, birth will be planned at a shorter time but marriage will be planned too. The marriage event will be executed only whether some properties (husband still alive, rates, etc.) are verified.

The **Rates** such as mortality, marriage, divorce, transition and fertility are still used and satisfy the same function.

Events flow directly from Socsim. To keep the simulation demographic-oriented and simple, no extra events were created. Some refactoring about groups and transitions were made to be able to label them instead of only giving them numerical identifiers.

With regard to the basic principles the following sections will explain in more detail how a *person* can be parameterized and which *events* are possible for that one.

6.2.3 Person

MiSca is intended to be as much parameterizable as possible. As persons are dynamic units, person's state (the set of its attributes) can change during the simulation. The change of these attributes can be observed, i.e. registering an action to be executed whenever the attribute state changes. The Table 6.1 regroups the main attributes for a person. The last column contains a mark if this attribute can be observed. As shown in this table, only the person's id as well as the person's parent ids cannot be changed and hence observed. The remaining attributes can be used to perform operations such as print statistics or trigger actions based on those attributes' changes, see Section 6.2.6.

Name	Description	Observable
id	Person's identifier	☐
group	Group where the person is affiliated to	☑
sex	Person's gender ("Male"/1, "Female"/0)	☐
mother	Mother's id	☐
father	Father's id	☐
birthdate	Birthdate (in months)	☑
marStatus	Historic of the person's matrimonial status (first is the actual one)	☑
(married, divorced, widowed, cohabiting, single)	Boolean attributes. They are, of course, consistent with each other	☑
siblings	Brother and Sister lists	☑
sibMother	Siblings lists from mother's side	☑
sibFather	Siblings lists from Father's side	☑
children	Children list	☑
parity	Number of children	☑
groupTransitions	Group transition history list (first is actual group	☑
relationList	Person's relation list (married, cohabitation)	☑

Table 6.1: Person's attributes

Method	Description
onMarriage(action)	when a new marriage event is triggered the action will be executed
onDivorce(action)	when a new divorce event is triggered the action will be executed
onDeath(action)	when a new death event is triggered the action will be executed
onBirth(action)	when a new birth event is triggered the action will be executed

Table 6.2: Register on type of events

6.2.4 Events

The microsimulation program MiSca inherits the same events as Socsim: **birth**, **death**, **marriage**, **divorce** and **transition**. The last one refers to transition from one group to another. It is similar to the Socsim model and is detailed in the section 6.2.5. Events are actions that are performed by people in the simulation but also actions that users[2] can generate during the simulation. Two main types of events exist:

- TimeStamp - This event contains a message that can be planned during the simulation. It is simply a particular event characterized by a message and a time.
- SimuEvent - This is an event characterized by a time at which the event is planned, a person planning it and the action to be executed at the specified time. This type of event is extended by the four types of vital events: **birth**, **death**, **marriage**, **divorce** and **transition**. Otherwise those are typically auto-generated events based on the *Scheduling Process*.
  According to the observable variables for a person presented in the last section, the following functions regrouped in the Table 6.2 allow users to observe person's events when there are executed. Those *onEvents* methods will execute all the stored `actions` when a new event of the corresponding type will be executed.
  Notice that the `action` argument is appended to the previous stored actions (if any). User can therefore store one action at a time or create a bulk of actions that will be executed at the same time.

[2] *User*: a non-programmer person which uses the program through the DSL. A *Programmer*: a person who has Scala background skills and uses the program in the core environment

Method	Description
onChange(action)	General function that can be used by the user to trigger actions whenever the group changes its status (new member in the group, member left, etc.)6.2.6
onIncrease(action)	Action will be executed when a person joins the group
onDecrease(action)	Action will be executed when a person leaves the group

Table 6.3: Register on Groups

6.2.5 Groups & Transition

Groups in MiSca can be defined by the user in order to model a grouping of people with common characteristics. For example, gathering persons from countryside in a group named *rural*. People can transit from one group to another. Each group represents a sub-population which is associated according to common characteristics. A group can represent anything such as: contaminated by a disease or not. A person can only be in a single group. This restriction is imposed in order to keep the rates table as simple as possible. Allowing multiple group membership would require to have a rate mixed by all groups of this person. Several trials were made to support multi-group membership but they result in a too much complexity for the user during the creation of the transition rates.

The Table 6.3 regroups all the observable methods that are possible for a specific group.

6.2.6 *onChange* operation

MiSca introduces a new flexibility concept in the microsimulation, the *onChange* operation allows the program's user to track and control a person's attribute, a group or an event during the simulation. Since programming languages appear unfamiliar for non programmer, a DSL was designed in order to provide the users with a specific and user-friendly domain language which is close to the natural language. For example the code in Listing 6.1 shows how to define a behavior to adopt when the status widowed of the person named Alain is changed. Since all the person's attributes can be observed, many combinations are possible allowing to create a variety of scenarios.

```
1 val Alain = Person()
2
3 Alain.widowed.onChange {
4   // Behavior to adopt when widowed attribute of Alain change
5 }
```

Listing 6.1: *onChange* operation performed on a group

Manipulate observers

In the last section we have seen that observers can be linked to one event, group or person's attributes. Those observers can be attached to a more general group of people for example the whole population or a group. The Listing 6.2 displays the usage of `onBirth` method. The second portion of code in this source-code shows how to count the number of births for the group called "Rural". The last portion counts the number of births for a specific person *p*.

```
1  /* Number of births for a population */
2  var countAll = 0
3  Population onBirth{
4     countAll = countAll + 1
5  }
6  /* Number of births for a the group Rural */
7  var countGroup = 0
8  Group("Rural") onBirth{
9      countGroup = countGroup + 1
10 }
11 /* Number of births for a p */
12 var countForP = 0
13 p onBirth {
14   countForP += 1
15 }
```

Listing 6.2: onBirth method example

6.2.7 Scheduling process

The MiSca's scheduling process was designed to be as much intuitive as in the real-life. People have plans: get married, give birth, etc., but other events might arise such as a divorce or a death which can lead to canceling or re-planning events previously planned for relatives. The person goes through a set of planning processes where each event has a certain probability to happen. The likelihood is computed from a rate defined by the state of the

person (matrimonial status, etc.) and the type of the event. These rates are stored in a rate table. Birth event has its own fertility rate table associated to female. Death, Marriage, Divorce events have also a rate table which can be parameterized according to the state of the person (see Section 6.3.6 for more detail about rate table). The living individual can plan one or "almost" all types of event (the term *almost* refers to the fact that if one type of event is *marriage*, you cannot already plan the divorce event, i.e. there are constraints on scheduling). The Figure 6.1 shows how events are planned for two persons during a simulation. The sub-figure (a) corresponds to the first step of a simulation: every living people, here $p1$ and $p2$, run through a scheduling process that sets events throughout their life. This is represented by the dotted red lines for each person. In sub-figure (b), once the events are scheduled (vertical lines) the first event is extracted from the queue which is $p2$'s event (symbolized by the black arrow). As Socsim used to do, every person during the simulation has a planned event. Therefore once the execution of $p2$'s event is executed, a planning operation is redone for $p2$. As shown in the sub-figure (b) the next planned event is $p2$'s death. The sub-figure (c) shows the next step in this event simulation which is the execution of $p1$'s event. Again a planning operation is performed at the end and one new event emerges. It is important to notice that during the person's life, new events can be planned alongside other previously planned events. In our case, the new event is planned after the next $p2$'s event (dotted red line in picture (c)). The last picture (d) displays the last execution of $p1$ which is the death event. The simulation will go through the same similar steps as explained above until there is no more event in the queue.

6.2.8 Graphical Interface

During the evolution of MiSca, two GUIs[3] were created to represent the simulation.

Embedded interface

This first solution consisted of using the Java's and Scala's *swing*[4] package to provide an user interface that could print most common-used demographic graphs such as population pyramid, gender distribution, etc. This first interface used *JFreeChart*[5], a Java chart library, that makes easy the creation of chart. The primal idea was to provide a toolbox to the MiSca software that was able to create charts based on all the attributes of the program. The graphs should use directly data from the variables to show in real-time

[3] Graphical User Interface: using JFreeChart, d3.js

[4] Java Swing:
`http://docs.oracle.com/javase/7/docs/api/javax/swing/package-summary.html`

[5] JFreeChart: `http://www.jfree.org/jfreechart/`

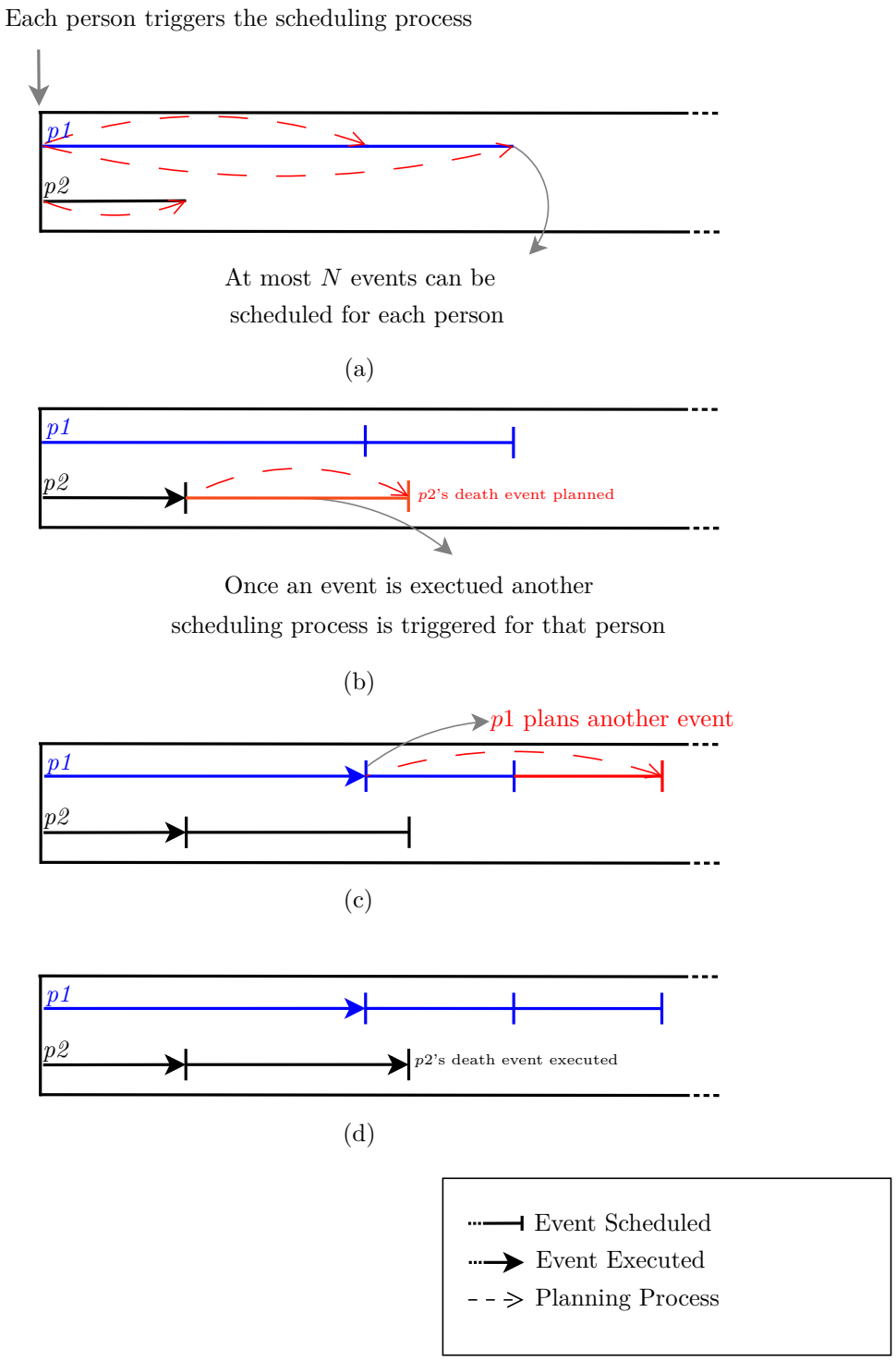


Figure 6.1: MiSca events

the evolution of the population. A screen capture of this interface can be found in the Appendix 10.3. Unfortunately the JFreeChart does not support well quick changes (e.g., refreshing quickly data to allow the graph to be updated). Therefore refreshing graphs with a lot of data has caused flickering on the screen[6]. With this restriction JFreechart imposes us to use only static graphs.

External interface

The second solution relies on an external web server that is hosted by MiSca. This server uses the d3js Javascript library[7] to generate graphs based on external data document such as CSV or JSON files. In order to be compatible with those data documents MiSca has methods that extract any listed data into JSON files. Therefore any kind of graphs can be created based on this data. Moreover tests on huge files reveals that this JavaScript library was faster than the JFreeChart package. Screen-shots of the interface regrouping the main menu and several graphs such as: population pyramid, gender distribution, population evolution can be found in the Appendix 10.4. Those graphs are dynamic, user can interact with them through the left and right arrow of their keyboard to move the time in the population pyramid graph. Other interaction with the mouse is possible to retrieve data of a current bar in the pyramid graph.

User can access, once the simulation executed, to the local website hosting the graphs through the address `http://localhost:8088`

6.3 Core Implementation

6.3.1 Core

Main Loop MiSca is based on a discrete event simulator with a priority queue containing events which are performed at a specified time. Many implementations of simulators are present in literature, our core is based on the Scala Book[12]. The events are stored in the priority queue which is chronologically ordered (most recent event is executed first). During the simulation, time is modeled by a sequence of integers, so it is not an actual "wall-clock" time. The Listing 6.3 shows the main loop of the program's core: the main loop executes each planned events until the queue is empty or the horizon is reached. The notion of *horizon* represents the ending time of the simulation. During the simulation, the `e.action()` line refers to the call of the `action` method

[6]The graphs loaded itself in approximately 2 seconds which is unmanageable in a real time system.

[7]D3: `http://d3js.org/`

of the event `e`. In order to fill the queue a *scheduling process* is running for each person at the beginning of the simulation.

```
1  def simulate(horizon: Int) {
2    curTime := Io.lastEventDate
3    population.foreach(_.randomEvent)
4      while (eventQueue.nonEmpty) {
5      val e = eventQueue.dequeue()
6      curTime := e.time.toInt
7      if (time <= (Io.lastEventDate + horizon)) {
8        e.action()
9      } else {
10       curTime = horizon
11       return
12     }
13   }
14 }
```

Listing 6.3: The main MiSca simulation loop

Scheduling Process To have a better idea of how the scheduling process is implemented, the Listing 6.4 contains the method `randomEvent`. In this code, events are planned with their respective `plan()` method and added in the priority queue with the `enqueue(randomEvent)` method. The planning consists of finding a *waiting time* for an event until a sufficent rate is reached (detailed information about waitingTime is at Section 6.3.2). The enqueuing method has two objectives: the first one is to store the event in the queue and the second one is to store the *call back* of the scheduling process (randomEvent).

The detailed enqueue function is shown in the Listing 6.5. The line `this.+(action)` handles that the Event (`this`) and the *call back* (randomEvent) are added to the Event Queue. This adding operation is performed by the method `waitDelay` (see line four of Listing).

For second objective, as said during the description of the simulation program, the scheduler is called back for a person after each executed event. Scala offers an elegant manner for achieving this kind of call back function: each event's enqueue function are defined as

```
def enqueue(action: =>Unit): Boolean
```

meaning that the by-name parameter `action` will be executed only when the action will be performed. This by-name parameter passing is convenient to our situation since the action is the `randomEvent` method itself. It will be called back after each event executed. The

```
1   def randomEvent {
2     if (deathdate == NotYetDefined) {
3       val death = DeathEvent(this)
4       death.plan() // Setting the waitingTime
5       death.enqueue(randomEvent) // Enqueue only if
        waitingTime set
6
7       val birth = BirthEvent(this)
8       birth.plan()
9       birth.enqueue(randomEvent)
10
11      val marriage = new MarriageEvent(this)
12      marriage.plan()
13      marriage.enqueue(randomEvent)
14
15      val divorce = new DivorceEvent(this)
16      divorce.plan()
17      divorce.enqueue(randomEvent)
18
19      for (gr <- groups if (gr != this.group)) {
20        val transit = new TransitionEvent(this, this.group, gr
          )
21        transit.plan()
22        transit.enqueue(randomEvent)
23      }
```

Listing 6.4: Scheduling process implementation

Figure 6.2 shows an example of a scheduling operation. In this figure the scheduling process has planned two events: a Marriage and a Birth event. Notice that, as in this case the action is `randomEvent`, the scheduling process will be called back. This `enqueue(randomEvent)` function is performed by each Person either at the beginning of the simulation(see the foreach code line three in the Listing 6.3) or either at the execution of its last event.

Waiting Time & waitDelay As previously mentioned, the planning of an event computes a *waitingTime*, i.e. a specific time to wait before the action is executed. Notice that if the Event's plan function does not return a time, that means no event of this type will be planned during this scheduling step. On the other hand, if the time is returned, the event will next be added in the priority queue, using the `enqueue` method. The implementation details about how to calculate a waitingTime is given in the Section 6.3.2

The Listing 6.5 shows the two remaining methods handling the storing of the events in the priority queue. The purpose of the `waitDelay`

```
1  def enqueue(action: => Unit): Boolean = {
2     this.+(action)
3     if (time > 0) {
4        waitDelay(this)
5        true
6     } else
7        false
8  }
9
10 def waitDelay(delay: BigDecimal, pers: Person)(action: => Unit
   ) {
11    waitDelay(SimEvent(BigDecimal(time) + delay, pers, x =>
      action))
12    }
13
14 // Store directly the event in the queue
15 def waitDelay(event: Event) {
16    eventQueue += event
17 }
```

Listing 6.5: Scheduling process implementation (cont)

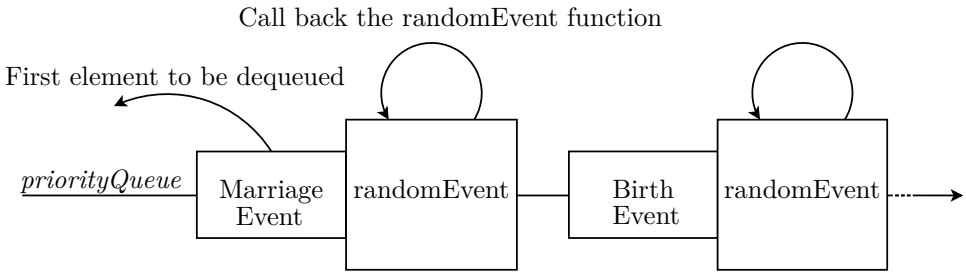


Figure 6.2: Scheduling process for a person

function is to add an action, associated with a time at which the action is supposed to happen, in the Event Queue. Notice that the association of this action and its time is encapsulated in the `Event` class before being pushed in the queue.

6.3.2 Generating waiting times based on rates

Events are scheduled according to specific rates and probabilities. Those rates (except births and transitions) are stored into a *RateMap* which is a hashmap where the key is composed of the person's attributes:

- Group
- Event Type
- Gender

- Marital Status

This key identifies a certain *situation* for one person. One parameter remains important to split the different rates: the person's age. Rates are separated into blocks called *ageBlocks*. Each block contains ranges of ages that are defined by the user. An example of ageBlock can be {[0 - 18], [19 - 40], [41, 70], [71, 100]}, where each range in the ageBlock has a specific rate which is a Double number associated to it. According to a key, the hashmap will return the ageBlock corresponding to a certain situation. This ageBlock will be sent to the method `computeWaitingTime` that will try to compute a waiting time with respect to the person's age (Notice that obviously only the range equal or superior to the person'age will be taken into account since planning event concerns only future events). The `computeWaitingTime` function is based on a *randomized experiment inside a loop*.

The Algorithm 1 displays the *computeWaitingTime* method. This function uses directly the Lee and Carter model as explained in the Section 2.4.1. Firstly, the rate corresponding to an age range is extracted from the *rateTable* and stored into a variable that we called here *currentRate*. From this rate, a duration needs to be calculated: this will be the time that the event needs to wait before being executed. The duration comes from the age range in the *currentRate* block. To better illustrates the `duration` the Figure 6.3 shows how duration is calculated. At the first iteration (inside the *while* loop), the duration is the difference between the upper age bound and the current age, which gives its current position in the age block (Represented in the first "wavy" arrow in the figure). If the event is not planned for this duration (i.e., the attempt based on the pseudo random number *logu* number fails) another loop iteration is performed. This time, the duration will be incremented by the size of the next age range (represented by the second wavy arrows). The looping operation will be done until either the attempt succeeds (i.e., the *logu* is below zero), or if there is no other range anymore (i.e., we reached the person's *AgeLimit*). In either case the corresponding *waitingTime* is returned and the event can be added in the Event Queue.

In order to understand the RateMap, a scenario is presented in the Figure 6.4: (1) a *female* of group *1* wants to plan an event *Marriage*. (2), the RateMap returns an AgeBlock corresponding to her situation, this block will be sent to the `computeWaitingTime` function that will try to plan a marriage according to the different rates given by the age range.

6.3.3 Sections

This concept is close to the Socsim's segments. Those are created by the user when he needs to change rates during the simulation. As the rate changes, all the planned events may become invalid since events are planned

Algorithm 1: computeWaitingTime method (scheduling operation)

Input: *ageBlock* received from the rateMap, *age* the person's age
Output: *waitingTime* the time where the action will be planned

```
currentRate ← ageBlock.getOrLast(age) ;
u ← a pseudo random number ;
logu ← the logarithm applied on u ;
ageAtEvent ← min(age, AgeLimit) ;
while logu < 0 do
    if ageAtEvent < currentRate.lowerAge then
        duration ← currentRate.ageInterval ;
    else
        duration ← currentRate.upperAge − ageAtEvent ;
    end
    temp ← logu/currentRate.rate;
    if currentRate.rate > duration then
        waitingTime ← waitingTime + duration ;
    else
        waitingTime ← waitingTime + temp ;
    end
    logu ← logu − (duration ∗ currentRate.rate) ;
    if ageBlock.next ≠ ∅ && logu <0 then
        waitingTime ← (AgeLimit − 1) − ageAtEvent;
        logu ← 0;
        return waitingTime ;
    end
    currentRate ← ageBlock.next ;
end
return waitingTime;
```

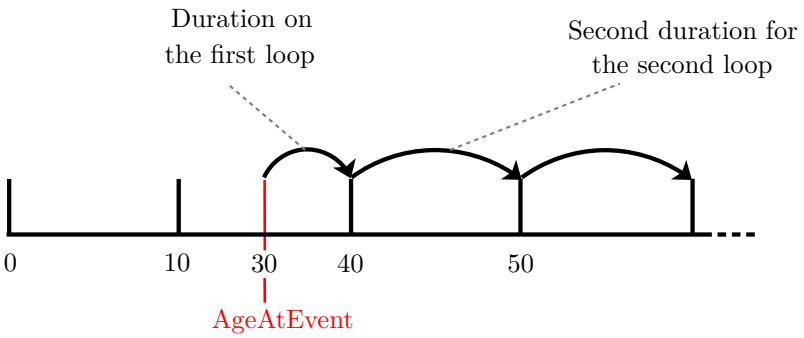


Figure 6.3: Calculating duration based on the person's age

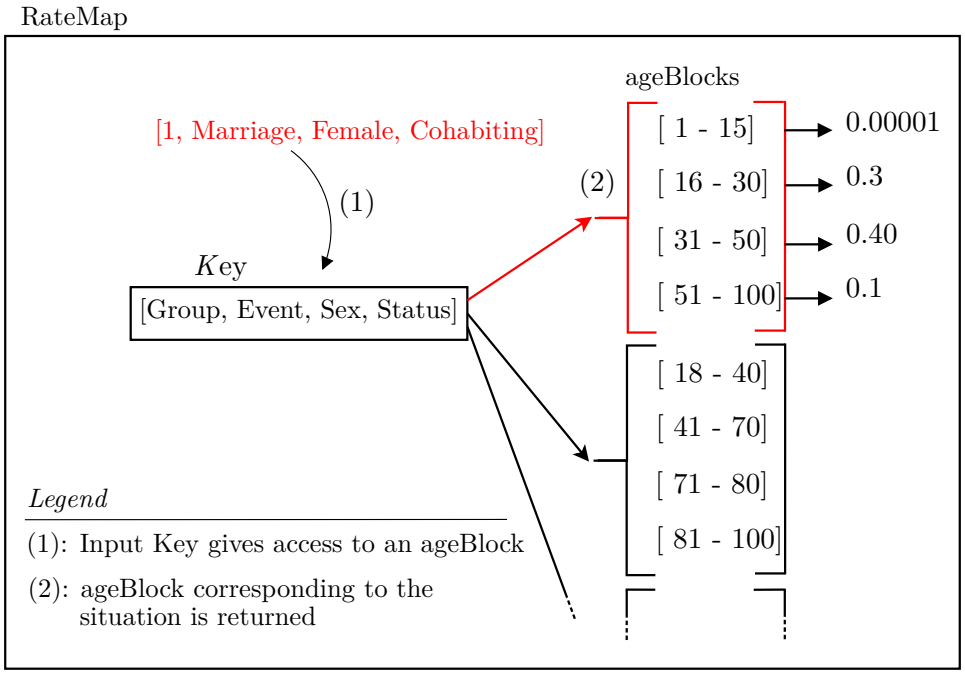


Figure 6.4: RateMap example with a Marriage Event

according to past rates. Socsim handles this rates change by allowing the user to modify it only between two segments. Moreover since events can be planned only inside one segment, Socsim ensures that all the events are coherent according to their associated rates. To keep this coherence MiSca relies on the same bases but allows multiples planning inside one segment. Several implementations were made in MiSca in order to check whether events were still applicable when user changes rates during the simulation but this checking operation raises too much processing for the simulation. Sections in MiSca can be viewed as multiple chained simulations, user can parametrized all the sections with specific rates and other parameters such as birth interval, duration, etc.

6.3.4 Events

Appendix 10.1 shows how the type of events are modeled in MiSca. Thanks to the object oriented mechanism, a main class called Event is created to regroup the common and necessary attributes to all the other types of event deriving from it. Those attributes are an *action*, and a *time* at which the action will be executed. There are two families of Event inside the simulation, the first one called *TimeStamp* is used by the programmer or the user to insert message during the simulation. The TimeStamp action is the printing of the message itself. The second family is *SimEvent* which concerns events related to a simulated person. This family is extended by all the events

listed below. This listing explains in more detail how those events are implemented and what are the particular case of each one. Since every event holds a particular case of execution (triggering only on certain conditions, specific `toString` method or `message` attribute), various functions such as the `plan()` function are overridden to deal with event's particularities.

Birth For this kind of event parameters such as mother's parity, fertility, time between childbirth, etc., must be taken into account to plan a birth. The `plan()` function is hence overridden to respect a planning associated with the parity and the fertility rate. The newborn lowers the fertility rate of the mother and rises the parity number. The simulation can plan one or more events of the same type in the future, however in this case due to the parameters such as fertility, mothers cannot plan several births during their life since the parameters associated with each birth will change.
For example in the Figure 6.5, the sub-figure (a) represents the scheduling process which plans a birth for a particular person $p1$ at a time 1320. The planning is based on parameters taken at the current time, the woman has no child hence the fertility multiplier is set to default (1.0). A problem arises when another event let say a marriage (sub-figure (b)) is executed at time 1220 and another birth event is re-scheduled at time 1400. This event is executed based on actual parameters therefore parity and fmult are set to 0 since the first birth event has not been performed yet. This planning will be executed based on wrong parameters assumptions. To avoid that, the execution of the second birth must be either re-validated by control operations (check whether the parity and the fertility rate allowed the birth at the time of the execution) or dropped. A trade-off has been made between accuracy and time complexity of processing such events: a woman cannot plan more than one birth at a time. With this assumption every planned birth will be executed and no control on their validity will be necessary anymore.

Associated to this first assumption, an extra parameter associated to the woman ensures that the time between birth is at least 9 months but can still be parametrized by the user. In MiSca, women can give birth to a child even if they are single, the child will be created but its father will remain unknown. User can change this default operation by tweaking the rates for the single woman by setting the fertility rate to 0 for example.

This kind of event involves the use of a specific rate map called *BirthRateMap* which is detailed in the section 6.3.6.

Death Since there is no specific conditions to plan a person's death, this class uses directly its parent super class to plan the death event. This

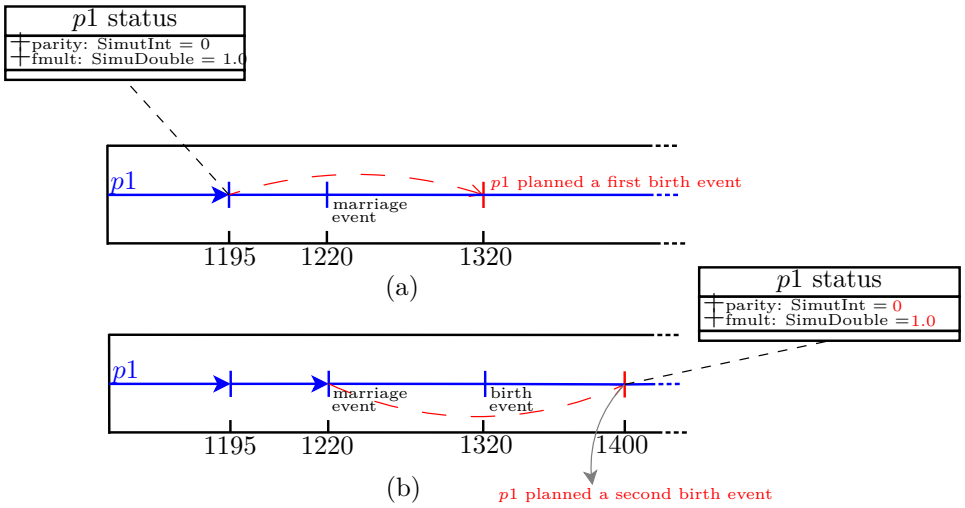


Figure 6.5: Birth event planning discussion

event uses the *RateMap* previously defined in the section 6.3.2. To prevent simulation from having persons which are older than the human life expectancy, the computeWaitingTime function returns always a maximum value of *AgeLimit - person's age*.

Cohabitation In Socsim as in MiSca, people in the simulation can create relation between them. We define relation as an union between a woman and a man. This relation is planned at a certain time and when it is executed this relation may become either a cohabitation event or a marriage event. The decision process is given in the Algorithm 2 and relies on a map called *CohabitationMap*. In this algorithm, the person who initiates the event rolls a die according to the rate extracted from the *CohabitationMap*. If the number is less than the rate, a cohabitation will be created between him and his spouse. On the other hand whether the trial fails the spouse may in turn throw the die. Finally if neither triggers a CohabitationEvent, the creation of a MarriageEvent will be performed. Notice that this algorithm is executed only when the *RelationEvent* is executed. Its plannification comes from the marriage plannification that is explained in the next point.

Marriage The marriage planning relies on the `plan()` method of the super class but controls before its execution must be performed:

- Age difference: test is performed to determine that this difference will not exceed the two boundary limits `marriageAgedifMax` and `marriageAgedifMin` (default is 20 years)

Algorithm 2: Decision process of a relation

Input: $cohabitationMap$ stores the rates
$group(S), sex(S)$ the person's and spouse's group and sex attributes

Output: $EventType$ return a MarriageEvent or a CohabEvent

$cohabiting \leftarrow person\ status$;
$cohab \leftarrow false$;
$prob \leftarrow cohabitationMap.get(group, sex)$;
$probS \leftarrow cohabitationMap.get(groupS, sexS)$;

if $cohabiting$ **then**
  $end\ cohabitation$;
  $cohab \leftarrow false$;
else
  if $prob \neq \varnothing$ **then** // Initiator rolls a dice
    if $rand < prob$ **then**
      $cohab \leftarrow true$;
    else
      $cohab \leftarrow false$;
    end
  else if $probS \neq \varnothing$ **then** // Spouse rolls a dice
    if $rand < probs$ **then**
      $cohab \leftarrow true$;
    else
      $cohab \leftarrow false$;
    end
end
if $cohab$ **then**
  return $CohabitationEvent$
else
  return $MarriageEvent$
end

- Endogamy/Exogamy: User sets the endogamy directive at 1 (all marriages are to be within group), -1 (all marriages are to be across groups) or 0(marriages can be done in or outside respective groups). Based on those parameters checks are performed to respect the user's specification.
- Incest: MiSca rejects parents, aunts, uncles, siblings and first cousins
- Previous couple: divorced people cannot be remarried together.
- Choose a spouse: this last operation searches for a possible spouse into a list of possible persons called the `marriageQueue`. By default, the marriage will be performed with the person's spouse (if cohabitant). If the person does not cohabit with anyone, a process of selecting a random spouse is performed. This process goes through the `marriageQueue` inspecting each person and attributes a *score* to them. This ranking algorithm is based on Socsim's marriage implementation. Spouses are preferred according to how close their differences match the `marriagePeakAge` value. If age difference is less than the peak value, then the score will be decreased. The idea behind this algorithm is to favor a certain optimal age difference, user can change the bounding value to prefer older or younger spouses. The default `marriagePeakAge` value is set to 36 months. Listing 6.6 shows the scoring function, in addition to the bounding value a `marriageSlopeRatio` is used to decline the score at a constant rate when the `marriagePeakAge` value is reached. In the implementation, the slope value is set to 2 meaning that the matches on the left of the peak (wife older than husband) decline in desirability twice as fast as age differences where the husband is older. If no spouse is found in the `marriageQueue` then this person is added to it and will wait for another person to be married with.

All this controls are performed and must pass before creating a marriage event. Hence a marriage can be planned but verification is done only when the event is executed in order to check the person's actual status. With this configuration many planned events may be ignored whether one control fails but marriage is at least as accurate (close to a real-life situation for finding a right partner) as possible.

Divorce / Separation A divorce occurs between two married people and separation occurs between two cohabitant. The planning of those relies on the rateMap. Due to the similarities present in the divorce and a separation, the implementation is performed in the person's `divorce` function. When this event occurs the instigator and its partner go back to their previous marital status.

```
def marriageScore(suitor: Person): Double = {
    val (woman, man) =
      if (sex == Female) {
        (this, suitor)
      } else {
        (suitor, this)
      }
    val maxS = 2
    val rightSlope = 0.0005
    val age_diff = woman.birthdate - man.birthdate
    if (age_diff - marriagePeakAge >= 0)
      maxS - rightSlope * (age_diff - marriagePeakAge) / 12
    else
      maxS - (rightSlope * marriageSlopeRatio) * (
      marriagePeakAge - age_diff) / 12
  }
```

Listing 6.6: Marriage scoring function

Transition This event has its own `plan()` function as it depends on a particular *Transition RateMap* detailed in the section 6.3.6. The rates are associated with each pair of groups representing the probability to move from one to the other. To lighten the user of creating rates of 0 from moving to the same group, we get rid of unnecessary events by allowing only transitions from different group. Group *inheritance* is also possible in MiSca. The inheritance can be parametrized from the user. The configuration can be either: from the mother, the father, the same sex as the parent or from the opposite sex of the parent. By default, as in Socsim, the child inherits his group number from the mother. In order to provide more flexibility to the user, MiSca offers another inheritance function that relies on probabilities. A group can be defined with two parameters driving the transmission decision. The two parameters are : `motherProbaGroupTransmission` and `fatherProbaGroupTransmission`. Those can be set by the user or have a default value set to 1.0 for mother and 0.0 for father. The decision is defined in the Algorithm 3.

As seen in Appendix 10.1 there are several classes that are not mentioned yet, those are *PlanningEvent* that are created with each events and stored directly into the person's history. This history was created to ensure consistency and to be able to make a census of people's live.

6.3.5 Person

A person is defined by the class `Person`. To keep the simulation clear we decided to store all the related actions associated to a person inside its own

Algorithm 3: Group inheritance function

Input: *mother father* of the child
Output: *Group* which the child will belong to
$prob \leftarrow$
$motherProbaGroupTransmission/(motherProbaGroupTransmission+$
$fatherProbaGroupTransmission)$;
$rand \leftarrow$ *a random number*;

if $rand > proba$ **then**
  return $mother'sgroup$
else
  return $father'sgroup$
end

class. The number of lines in this class may appear huge but a trade-off has been made to keep the use of references inside the method parameters as low as possible. Moreover some automated functions are also linked with the creation of a person. Actions like birth trigger the creation of a new individual where the methods `updateParents()` and `updateSiblings()` are automatically triggered. Those two methods handle the updating of the parents' information. We use the concept of *Companion Object* to associate to each person an unique id that is incremented when a person is created.

6.3.6 Maps

This section explains the different types of maps that are present in the software, we use hashmaps to implement them. The choice of hashmaps is motivated by the time complexity of accessing or storing one element which is $\mathcal{O}(c)$ (where c is a constant) depending of the map maximum length and the hash keys distribution. As previously explained, the scheduling process uses a rate map to plan events. This *RateMap* is shown in the Figure 6.6. Specific accessing methods were created to retrieve information based on constructed key. The key is formed with relevant information specific to each maps. The *RateMap* uses a key of type [Group, EventObject, Sex MarStatus] which are necessary to classify for each group, event, gender and status a specific rate. But other maps like *BirthMap* relies only on the [Group, MarStatus, Parity] to deal with birth rates. Nevertheless generic methods listed below are created to simplify the access to the information:

get(k) this is a common accessing method to retrieve values based on the map's specific key.

getOrLast(k) sometimes during the simulation, returning no information may not be an option, choices must be made to retrieve the closest

relevant information if the most accurate one is not available. For example, in the case of a birth for a married woman of group 2 with a parity of 2. If the most appropriate rate in the table is not found, this function will be self-called with a slightly different key (typically decrement one parameter). The key will be [Group(1),Married, 2] where the group sub-key has been decremented by 1 since no rates have been found for the group 2. The complete key decision algorithms for all the types of maps are given in the Algorithms 4 , 5 and 6. Note that in some case, no result can be found (`None` is returned). In this case the rate is set to zero meaning that there is no chance of triggering this event. The RateMap and BirthMap algorithms work the same way. First the key is checked into the map, if no value is returned the `getOrlast` method will be self-called with decrementing one parameter in a priority order. The key decision algorithm is based on default value defined in Socsim. This allows the user to not defined rates for any combinations of keys.

The priorities for the *BirthMap* are:

1. **Decrement the group id** This is the first transformation of the key, the decremented group's key will be retried until the group id reaches 1 (the last possible one). If no rates are found for those upper groups, the next step will be to decrement the parity.
2. **Decrement the parity** Similarly the decremented parity will be tried until reaches zero.
3. **Change the marital status** Whether no rates has been found yet, the algorithm will try to change the marital status to the "closest" one (i.e., another status that corresponds the most to the person): Divorced status becomes Single, Widowed becomes Divorced and Cohabiting is changed to Married.

And for the *RateMap*:

1. **Decrement the group id** until group 1 is reached.
2. **Change the marital status** Similarly a descending chain is created to perform a trial and error on the key: Cohabiting → Married → Widowed → Divorced → Single → None.

+(kv) Updates the map with an new key-value mapping.

keys Returns a collection of the keys present in the map.

As *BirthMap* and *RateMap* uses specific accessing methods(getOrLast, etc.) based on the key, the *TransitionMap* does not require such accessing methods since the rates must be mandatory in order to go from one group to another. Same rule applies for the CohabitationMap that is used in the

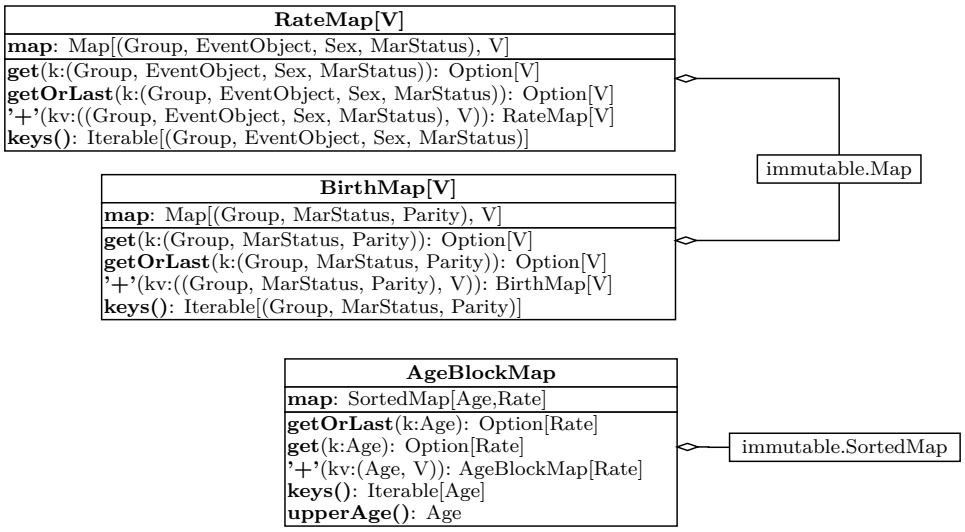


Figure 6.6: MiSca maps

decision process of determining which relation ends up with a cohabitation event. No extra methods are therefore necessary. Thus the TransitionMap and the CohabitationMap rely on a traditional Scala Map.

6.3.7 SimuTypes

The MiSca's flexibility has been built upon the use of *observer*. An observer is a one-to-many dependency between objects so that when one object changes his state, all its dependents are notified and updated automatically[2]. In other words the observer pattern defines and maintains adependency between objects. To ensure a maximum flexibility for a simulation, the user needs to observe lots of parameters associated to a person, a group, and defining a specific observer for each parameters could be dreadful for both user and programmer. To ensure an efficient observer creation, a redefining operation must be applied on primitive types. With those redefined types called *SimuTypes*, user creating variables or using existing ones have already an observer available for every variable in the program.

SimuInt

The file `SimuTypes.scala` regroups all the redefined types, as an example the Listing 6.7 presents the SimuInt: this class is used to create an observable Integer with new operations that can be applied on it. Other simulation type such as SimuDouble, SimuBoolean are very similar to the SimuInt construction, therefore, we will cover only this one. Besides those primitive types a SimuList is also implemented to control population list such as

Algorithm 4: BirthMap's getOrLast method

```
Input: key triplet (Group, MaritalStatus, Parity)
Output: value closest value matching the key

if birthMap contains (key) then
    return value
else                                  // Change one parameter of the key's triplet
    if birthMap keys ≠ ∅ then
        if Group == 1 and Parity == 0 then
            // Try another status
            switch MaritalStatus do
                case Single
                    None
                case Married
                    None
                case Divorced
                    getOrLast(Group, Single, Parity)
                case Widowed
                    getOrLast(Group, Divoced, Parity)
                case Cohabiting
                    getOrLast(Group, Married, Parity)
            endsw
        else if Group == 1 then
            // Try another parity
            getOrLast(Group, MaritalStatus, Parity−1)
        else if Group < 0 then
            None
        else
            // Decrement the group id
            getOrLast(Group −1, MaritalStatus, Parity)
        end
    else
        None
    end
end
```

Algorithm 5: RateMap's getOrLast method

Input: *key* triplet (Group, EventObject, Sex, MaritalStatus)
Output: *value* closest value matching the key

```
if rateMap contains (key) then
    return value
else                                  // Change one parameter of the key's triplet
    if rateMap keys ≠ ∅ then
        if Group == 1 then
            // Try another status
            switch MaritalStatus do
                case Single
                    None
                case Married
                    getOrLast(Group, EventObject, Sex, Widowed)
                case Divorced
                    getOrLast(Group, EventObject, Sex, Single)
                case Widowed
                    getOrLast(Group, EventObject, Sex, Divorced)
                case Cohabiting
                    getOrLast(Group, EventObject, Sex, Married)
            endsw
        else if Group < 0 then
            None
        else
            // Decrement the group id
            getOrLast(Group −1, EventObject, Sex, MaritalStatus)
        end
    else
        None
    end
end
```

Algorithm 6: AgeBlockMap's getOrLast method

Input: *key* age
Output: *value* closest value matching the key

if *ageBlockMap contains (key)* **then**
  return *value*
else // Try to find an upper key
  $upperKeySet \leftarrow (ageBlockMap.keys > key)$;
  if $upperKeySey \neq \varnothing$ **then** // Try with an upper key
    ageBlockMap contains ($upperKeySet$)
  else if *ageBlockMap keys* $\neq \varnothing$ **then** // Try with the biggest map's key
    ageBlockMap contains (ageBlockMap max key)
  else
    None
  end
end

brother or sister associated to each person. Two kinds of lists exist the Sorted and Unsorted one detailed in the Section 6.3.7

The table 6.4 regroups all the parameters or methods that can be applied with the new observable Integer. Since the observer must be triggered when a change occurs, the class must hold the last value affected to the Integer and the new one. By comparison of the two values inside the type, actions are executed if there is a difference between the old and the new one. Methods provide to the user of the `onChange` operation, more control on the type (`increased`, `decreased`, `delta`).

Transferring actions An important point of the refined operators such as `":="` is to keep the actions while affecting the SimuInt's variable. This operator has to be redefined because Scala doesn't allow to override the default `"="` affectation operation. Since SimuInt is a class, a user that types the last line in the following example will erase all of his actions previously stored in the SimuInt `myInt` variable.

```
1  var myInt = SimutInt(10)
2  var counter = 0
3
4  myInt.onChange{
5     if (increased) {
6        /* Action */
7        counter = counter +1
8     }
9  }
10
11 myInt = 11 /* Causes a change of the reference of the
```

```
1  class SimuInt(i: Int) {
2    var value = i
3    private var actions: List[() => Unit] = List()
4    var oldValue: Int = 0
5    def onChange(action: => Unit) {
6      actions = (() => action) :: actions
7    }
8    def :=(v: Int) = {
9      if (v != value) {
10       oldValue = value
11       value = v
12       actions foreach (_())
13     }
14   }
15   def increased = value > oldValue
16   def decreased = value < oldValue
17   def delta = value - oldValue
18   def ==(i: Int) = value == i
19   def ==(i: SimuInt) = value == i.value
20   def !=(i: Int) = value != i
21   def !=(i: SimuInt) = value != i.value
22   override def toString = value.toString
23 }
```

Listing 6.7: SimuInt class

```
SimuInt class */
```

To overcome this problem, user **must** use the ":=" at each time when an affectation must be made. The correct affectation will be: `myInt := 11` and the result of this affectation will trigger the incrementation of the `counter` variable.

onChange This method is common for all the SimuTypes and is defined as follow:

```
def onChange(action: => Unit) {
    actions = (() => action) :: actions
}
```

As user can register to an observer, he can provide an action which is a piece of executable code that will be executed during the changing state of primitive type. Severals actions can be associated to each type hence the variable `actions` is implemented as a list. When the type's state changes all the action will be executed.

Operators / Methods	Description
:=	affectation operation
onChange	used to record to the SimuInt observer
increased	boolean value, check if the new value entered is bigger than the last affected
increased	boolean value, check if the new value entered is lower than the last affected
==(i: Int)	equality comparator between the candidate integer and the SimuInt's internal value
==(i: SimuInt)	equality comparator between the candidate SimuInt and the internal value
!=(i: Int)	difference operator between the candidate integer and the SimuInt's internal value
!=(i: SimuInt)	difference operator between the candidate SimuInt and the internal value

Table 6.4: SimuInt operators and methods

SimuSortedList

Sometimes, an ordered list can be useful for the programmer and for the DSL's user, this is the case for children, siblings, etc. Lists are ordered based on their comparator method associated to the type of the element in this list. A comparator is introduced in the Person's class to compare each individuals. In order to do such comparison *SimuSortedList* and the Person's class extends the `Ordered[Person]`. Groups have the same extension and can be inserted in a sorted list with the comparator based, here, on their group number (lowest id first). The Group's companion object holds the default comparator (id-number based) but can be redefined by the user with the method `compareWith(comparison: (Group, Group)=>Int);`. The Listing 6.8 shows how the SimuSortedList is implemented. Some operators related to the generic lists were overridden: `++`, `::`, `tail` to keep or to transfer the actions from a variable to another(Recall from SimuInt transferring actions).

```
1  class SimuSortedList[A <% Ordered[A]](list: List[A]) {
2    var value = list.sorted
3    private var actions: List[() => Unit] = List()
4
5    def this() = this(List())
6
7    def onChange(action: => Unit) {
8      actions = (() => action) :: actions
9    }
10
11   def :=(list: SimuSortedList[A]) {
12     value = list.value
13     actions foreach (_())
14   }
15   def ++(v: List[A]) = {
16     var newList = new SimuSortedList(value)
17     v.foreach(pers => newList = pers :: newList )
18     newList.actions = actions
19     newList
20   }
21
22   def tail = {
23     val tempValue = value.tail
24     val newList: SimuSortedList[A] = new SimuSortedList(
       tempValue)
25     newList.actions = actions
26     newList
27   }
28
29   def ::(pers: A) = {
30     def insertInList(list: List[A]): List[A] = {
31       if (list.isEmpty)
32         List(pers)
33       else {
34         if (pers >= list.head)
35           pers :: list
36         else
37           list.head :: insertInList(list.tail)
38       }
39     }
40     val tempValue = insertInList(value)
41     val newList: SimuSortedList[A] = new SimuSortedList(
       tempValue)
42     newList.actions = actions
43     newList
44   }
45   override def toString = {
46     value.toString
47   }
48 }
```

Listing 6.8: SimuSortedList class

6.4 DSL

This section presents the domain specific language for demographic simulation designed on top of the Scala programming language. As said in Section 2.2, Scala provides interesting features to build such DSL. This facilities are mainly due to the functional and object-oriented paradigms perfectly mixed in Scala.

In this section, we first present major facilities proposed by MiSca. The major facilities provided by MiSca are presented in a kind of user manual. Then we discuss the domain-specific semantics built on top of Scala, along with some brief implementation details in order to present how Scala facilities can be used to design domain specific languages.

6.4.1 User Manual

Before exploring the semantics implemented, comes a presentation of all features provided by MiSca to configure and run a simulation.

MiSca is used in some similar ways to Socsim. In order to maintain a kind of compatibility with it, MiSca uses input and output files with format similar to the Socsim one.

Input and output files

As microsimulation simulates *demographic events of individuals*, it needs a lot of rates depending on age, sex, group, marital status or parity. Moreover, these rates have to be specified for different types of events (death, birth, marriage, divorce and group transition). In addition of this data, simulation needs an initial population, list of marriages and transitions. This results in a large amount of data. Such data are stored like Socsim does.

MiSca reads and writes population data in population *.opop* files, marriage *.omar* file and transition *.otx* files.

As these files are identical to those used by Socsim, we will present them briefly. More information about this files is available in the document *Socsim Oversimplified*[5]. The information has the following format:

Population The population file records the initial population of the simulation. If some individuals of the initial population are married, then a marriage file recording these marriages is required. The population file is generally suffixed with *.opop*. The structure of this file has already been described in the Section 3.1.1 in the Table 3.1.

Marriages The marriages file records the marriages experienced by the initial population. The marriages file is generally suffixed with *.omar*. The structure of this file has already been described in the Section 3.1.1 in the Table 3.2.

Transitions The transitions file records the transitions experienced by the initial population. The structure of this file has already been described in the Section 3.1.1 in the Table 3.3.

Rates The rates file contains the vital demographic rates. A set of rates for a demographic event starts with an header line such as:

birth 1 F single 0

This header line indicates that follows the definition of a set of age specific fertility rates for individual of group 1 female single and with parity 0. This header is followed by a series of lines defining rates. These lines are composed of three numbers: the first two indicates the upper age bound. The first one is the year and the second one is the month. Any age category can be defined. The third number is the rate itself. It is a rate per month. The Listing 6.9 shows an example of such a rate block.

```
1   death 1 F single
2   0     1   0.088135
3   1     0   0.0083525
4   5     0   0.0014636
5   10    0   0.00034839
6   15    0   0.000191579
7   100   0   0.0590448
```

Listing 6.9: Rate block example

This example specifies six age categories. Individuals female, single of group 1 have rates for death event defined as: the rate per month of death is 0.088135 between 0 and 1 month, 0.0083525 between 1 month and 1 year, etc. And finally the rate per month of death is 0.0590448 between 15 years and 100 years. Notice that the upper age bound is exclusive in the age interval.

An example of such rates file is provided in Section 10.1.5.

Fortunately the user does not have to specify rates for all combinations of characteristics. Default rules are defined for missing combinations. This default rules are described in *Socsim Oversimplified*[5] and are integrated in the implementation of the rate maps described in Section 6.3.6.

The population, marriages and transitions files output by a simulation can of course be used as input of another simulation.

Simulation parameters

MiSca provides parameters that can be set up in order to configure a particular simulation. This section aims to list them and to present some interesting aspects of this configurable settings. Here is the minimum mandatory ones :

`inputFiles("input/")` You can specify the folders from which you want to load the input population, marriages, transitions and rates. This folder needs to be created beforehand.

`outputFiles("ouput/")` You can specify the folder into which you want to save the ouput population, marriages and transitions. This folder needs to be created beforehand.

`loadPopulation("populationFile.opop")` Reads and creates the population recorded in the file specified in argument.

`loadMarriages("marriagesFile.omar")` Reads and creates the marriages recorded in the file specified in argument. Notice that it is mandatory to load population before loading marriages.

`loadRates("HIVscenario.rate")` Reads and creates the rates recorded in the file specified in argument. It initializes the required simulation rate tables.

`Simulation simulate 250.years` Defines the length of the simulation. It has to stand at the end of the configuration lines because it is the instruction launching the simulation.

Of course, default values are defined for all these settings. But once you change a setting, this new value is used during the whole simulation. You can also change permanently these settings in the package object resources.

Be aware that the sequencing of this instructions is important. Thinks of these instructions as an actual execution. For example you first load and create the population before loading the marriages because marriages involve people.

Then some configuration settings can be defined. Most of them come from *Socsim* and are explained in [5]:

`propMales` Specifies a sex distribution for the simulation, more specifically the proportion of birth giving a male. Default values is 0.5112.

hetfert "Enables the heterogeneous fertility feature (0 to disables it). If enabled, each female will have a beta distributed random variable by which her fertility is multiplied before random waiting times are generated. The result is a wider variation in sibling set size than would otherwise result. Note that the degree to which this fertility 'multiplier' is inherited by daughters can be set by the user (see **alpha** and **beta**)."[5] . The default value is 1

alpha and beta They determine "*the degree to which the fertility multiplier is inherited from each woman's mother*"

minBetweenBirth This is the minimum time interval required between two birth. Its default value is 9 months.

marriageInGroup This specifies the behaviour of persons looking for a spouse with regards to other groups. There are three behaviours: endogamy (i.e. the person only chooses his/her suitor in the same group), exogamy (i.e. the person only chooses his/her suitor from a different group) and random (no matter the group membership of the suitors). Three constants are thus defined `Endogamy`, `Exogamy` and `Random`. The Default value is `Random`.

marriagePeakAge Specifies the spouse age difference prefered when choosing a spouse. The default value is 36 months.

marriageSlopeRatio Used with marriagePeakAge to compute a preference score when choosing a spouse. The default value is 2.0.

marriageAgedifMin Defines the minimum age difference between spouses (husband age - wife age). The default value is 20 years.

marriageAgedifMax Defines the maximum age difference between spouses (husband age - wife age). The default value is 20 years.

childInheritsGroup Determines group membership inheritance at birth. It can be:

- `FromMother`
- `FromFather`
- `FromSameSexParent`
- `FromOppositeSexParent`

The default value is `FromMother`. The group inheritance can also be defined specific for a group, see Section 6.4.2 for more information about group inheritance.

And finally one can create and combine different entities such as persons, groups, etc. Here is a description of two of such elements:

`ChainedSimulation` and `Section` In order to provide a facility similar to Socsim segments, we decided to keep the simulation implementation as is, but we added a surrounding abstraction that can contain chained simulation units. This way, one can specify a given set of vital rates effective under a particular section of a simulation. One can then chain this section by another section while defining a new set of rates effective for this new section. One is also able to save population, marriages and transitions after a simulation section unit. The advantage of this implementation is its modularity aspect. The simulation core is not modified and is kept quite simple. The Listing 6.10 shows an example of use of a Chained Simulation. This example extends the HIV scenario of the running example in Section 5. In this example we define two sections that load two different rates sets in order to expose people to two different event risks. Moreover, at the end of each section we save the current population in a specified file. Observe that a section body has the same form as a simulation body. Indeed, it is a simulation that one can chain with others. The sections are executed in their definition order.

`Group("group-name") | Group(1)` A semantic is provided to define a group:

```
Group("HIV-positive")
```

A group is created either by specifying an integer id or by a name label. Notice that groups have to be declared before loading the population because id are automatically and sequentially attributed at the time of group declaration. So if an initial population from a .opop file defines group 1 and 2 (with group 2 representing HIV-positive group), these 2 groups will be created at the population loading and the id 1 and 2 will be attributed. Then if comes a `Group("HIV-positive")` declaration, the first id not yet attributed will be used for this group (i.e. 3 in our example). However if one actually wants to load the population before declaring groups with label, we provided the following facility : **`Group("HIV-positive", 2)`**, this actually affects label "HIV-positive" to group with id 2.

6.4.2 Domain-specific semantics

Designing a good DSL involves a delicate trade-off between expressiveness and communicativeness in one hand, and the simplicity and genericity on

the other hand. Indeed, it is quite easy to "hardcode" some features to implement a common english language. But this can lead to some complex, not easy to learn and not generic DSL.

An important principle is that a well-designed dsl is based on a well-designed problem abstraction.

Using the DSL to write scenarios is equivalent to write Scala code, *based on the semantics that MiSca implements* for designing scenarios.

For the sake of brevity, this section will not cover the whole implementation details. Rather, we attempt to illustrate each feature used to implement new semantics.

DSL implementation is driven by the need of facilities to express some rules. Some of them will be explored in the following sections. This will be an opportunity to present some Scala features very helpful in designing DSL.

Observers

We already explained the use of observers on almost all attributes of a person. For a recall, this enables to put some action in the observer list of an attribute. These actions will be executed when the attribute changes. Please refer to the table 6.1 to see all observable attributes. For that purpose, we implemented *observable types* to substitute common types. For example we substitute `Int` by `SimuInt`, `Boolean` by `SimuBoolean`, `List` by `SimuList` or `SimuSortedList` for sorted list, etc. See Section 6.3.7 for more details.

The vital events (birth, death, etc.) are also observable. This means that you can add some actions to be executed when an event is executed. The HIV scenario example, in Listing 5.1 in Section 5, shows such a case. Please refer to Section 6.2 for more information about observer on events.

This functionality allows the user to have control on almost all points of a simulation. For example, the user is able to reduce mother's fertility when her child dies.

There is three levels of use of such observers on events:

On individuals This one is applied on an individual. It should rarely be used because we are rarely interested to observe a particular person in microsimulation.

On groups This one is applied on each member of a group. So this observer is "dynamic", i.e., when an individual leaves a group, he is not anymore subject to this observer. This can, for example be used as in the example of the HIV spreading in Listing 5.1.

On all the population This one is applied on each person of the actual and the future population. This can, for example, be used to count the number of birth, as in Listing 6.11:

```
1 val numbBirth = 0
2 Person.onBirth{numbBirth = numbBirth + 1}
```

Listing 6.11: Observer on Population example

Observers on events are simply resolved by a method in the `Person` class. That method retrieves in the three levels (the individual's actions, his/her group actions and the population related actions) all the actions to be executed when the individual experiences the corresponding event. For that purpose these three entities contain a list of actions for each vital event.

Group DSL

At each time in a simulation, each individual of the population belongs to exactly one group.

Groups appear to be very helpful as a way to extend easily the model.

In addition to an id, a group can be identified by a label. When using this statement, either the group is created and returned if it does not exist yet, or it is just returned if it exists. Here is how groups can be used:

- `Group(1)` This will return the Group 1 (and create it if it does not exist already).

- `Group("blue eyes")` This will return a Group named "blue eyes" (and create it if it does not exist already).

If no label is specified, a default label is affected. It simply corresponds to the id translated in a character string.

Moreover, we extended groups to be able to specify some rules on them. For example, we can add some group inheritance specific to a group. The semantics implemented for this purpose is explained in the following section.

Another facility provided by this semantics is that the user can define new rates applicable to a group for a combination of person's characteristics. The Listing 6.12 shows an example of use of this rates definition semantics. These rules will update the rates tables. Notice that this can overwrite existing rates in the rates tables (remember that this rates tables are implemented as maps whose keys are group, parity, marital status and age).

```
1 Group("VIH") { group =>
2   death_rate of Single male group up_to 30.years is 0.7
3   death_rate of Single male group up_to 60.years is 0.7
4   divorce_rate of Divorced female group up_to 40.years is 0.6
5 }
```

Listing 6.12: Rates defintion semantics

Inheritance and Transmission DSL

An important configuration in the MiSca model is the group inheritance. At birth, a child can inherit from mother (by default) father, same sex parent or opposite sex parent.

As we said in Section 6.4.1, inheritance of group membership is defined by the `childInheritsGroup` setting. MiSca is provided with a semantics to express in a more convenient way powerful inheritance rules. Here is some examples:

Global inheritance The semantics allows to express that all the population inherits from a particular parent, for example:

```
People inherit_from father
```

Group specific The DSL also provides facilities to define how mother and father transmit their group to children. Listing 6.13 presents such an illustration.

```
1 Group("Rural") inheritance { group =>
2   mother transmits group with_proba 0.7
3   father transmits group with_proba 0.6
4 }
```

Listing 6.13: Group inheritance example

Chained simulations

As explained previously, MiSca provides a feature similar to Socsim segments. This is resolved with a chaining of simulation. It was chosen to surround the simulation abstraction by another abstraction:

`ChainedSimulation`.

This allows to keep the simulation implementation simple and to avoid to tempt to modify a well-designed abstraction at the risk of introducing errors.

The Listing 6.10 in Section 6.4.1 gives an example of use of the chained simulations.

The Listing 6.15 shows the implementation details of this semantics. This one is pretty simple. A class **`Section`** abtracts the section concept. It only contains an attribute of type unit that represents the simulation section instructions. This one is passed by name, allowing to be passed without being evaluated right now. This class also contains a method named **`process`** to execute this simulation.

The **`ChainedSimulation`** itself is an object that extends the **`Simulation`** class. This object contains a list of sections that are added thanks to the inner object **`section`** and its **`apply`** method which also takes a passed-by-name parameter of type Unit. This structure allows to define a section environment to include all section rules and instructions.

In the same way, a **`ChainedSimulation`** contains an **`apply`** method that take a Unit as parameter. Observe that this parameter is not used. It is only used as environment for the section definitions. This **`apply`** method is still used to execute the **`simulate`** method that iterates over all the sections in order to run them. Note that the **`eventQueue`** containing all the events of the simulation, is emptied after each section execution because if a new set of rates are defined in the following segment, events previously scheduled will not be applicable with regards to the new rates.

This implementation allows to write chained simulation with a clear structure, such as listed in Listing 6.14.

Duration DSL

MiSca implements a duration semantics to enable the user to write some duration in the following format :

```
6.years and 3.months
```

instead of writing

```
6*12 + 3
```

The first expression is highly more expressive. A quick look is enough to understand it represents a duration of 6 years and 3 months. No reflexion needed. Think of how productivity and communication can be increased with such features.

The time unit used in all the implementation is the month. So when one writes such an expression, it is translated into an equivalent number of months (i.e. 75 months in this case) thanks to an implicit conversion defined in the `Duration` object. This implicit conversion opens the `Int` class in order to add some duration related methods to it. Notice that one of the major advantages of the Scala opening class system is that it is not global

(unlike other such common systems, such as Ruby). This means one needs to explicitely import the implicit conversion where one wants to use it. In other words, it is up to the user to define the context of the program.

The `Duration` class also allows the user to transform a duration in a string properly formated. This is provided by the `format` method. Notice that it takes care about the singular or plural form of the words 'year' and 'month' depending respectively on the number of years and months.

The Listing 6.16 shows the implementation details of this semantics.

Settings configuration

Opening an existing class to "add" new methods by means of implicit conversion is also used to change value of the simulation settings in a more literal way. For example if one wants to change the default value of the `marriageInGroup` to make the model endogamous, one can write:

```
marriageInGroup is Endogam
```

This is done just by defining an implicit conversion from the `Int` to a builder class, and defining a method named `is` in this class.

A significant part of the DSL implementation is based on this facilities in addition to other one provided by Scala. For example, the nonnecessity of using '.' when calling a method on an instance, combined to the possibility to remove parentheses makes the syntax more confortable. The ability to redefine operators (e.g. '+') in a particular context is also valuable.

```
object ChainedSimulationExample extends App {

  Group("HIV-negative")

  Group("HIV-positive")

  loadPopulation("HIVscenario.opop")
  loadMarriages("HIVscenario.opop")

  ChainedSimulation {
    // section 1
    section {
      loadRates("HIVscenario.section1.rate")
      val HIVTransMult = 0.5
      Person onMarriage { person =>
        if ((person.spouse is "HIV-positive") || (person is "
        HIV-positive")) {
          tmult of person.spouse is HIVTransMult
          tmult of person is HIVTransMult
        }
      }

      Group("HIV-positive") onTransition { person =>
        person infects tmult to person.spouses
        Simulation deleteEventsOf person.spouse
        Simulation randomEventFor person.spouse
      }
      Simulation simulate 50.years
      savePopulation("HIVscenario.out1.opop")
      saveMarriages("HIVscenario.out1.omar")
    }
    //section 2
    section {
      loadRates("HIVscenario.section2.rate")
      val HIVTransMult = 0.7
      Simulation simulate 100.years
      savePopulation("HIVscenario.out2.opop")
      saveMarriages("HIVscenario.out2.omar")
    }
  }

  population.foreach(x => x.history)
}
```

Listing 6.10: Chained Simulations Example

```
ChainedSimulation {
  section {
    // ...
  }
  section {
    // ...
  }
}
```

Listing 6.14: Structure of the chained simulation semantics

```
object ChainedSimulation extends Simulation {
  var sections: List[Section] = List()

  def apply(chainedSimulationCode: Unit) {
    simulate()
  }

  object section {
    def apply(simulation: => Unit) = {
      sections = sections :+ new Section(simulation)
      this
    }
  }

  override def simulate(horizon: Int = 0) {
    for (section <- sections) {
      section.process()
      eventQueue = new PriorityQueue[Event]()
    }
  }
}

class Section(simulation: => Unit) {
  def process() = simulation
}
```

Listing 6.15: Implementation of the chained simulation semantics

```
class Duration(duration: Int) {
  def years = duration * 12
  def year = years
  def months = duration
  def month = months

  def years_1month = duration * 12 + 1
  def and(months: Int) = this.duration + months

  def format: String = {
    val (years, months) = (duration / 12, duration % 12)
    val yearsString =
      if (years == 0 || years == 1)
        s"${years} year"
      else
        s"${years} years"
    val monthsString =
      if (months == 0 || months == 1)
        s"${months} month"
      else
        s"${months} months"

    s"${yearsString} ${monthsString}"
  }

  def toTuple = (duration/12, duration%12)
}

object Duration {
  implicit def Int2Duration(duration: Int) =
    new Duration(duration)
}
```

Listing 6.16: Duration semantics implementation

Chapter 7

Validation

7.1 Method

To validate this new software, we first decided to compare our results with the Socsim tool since the program is based on it. As we will see, results are quite similar but some variations appear. We will explain what are the leading causes of those differences. However, to validate our model we must first ensure that the basic principles, specification and properties are respected during the whole simulation. This first coding validation relies on test methods based on the ScalaTest suite. Some tests of the MiSca's main concepts will be explained in order to demonstrate how such tests validate the specifications during the simulation. Once all these unit tests are checked, the new software can be validated with Socsim.

7.1.1 Unit Test methods

In order to validate our code, we have used ScalaTest[18]. This testing tool provides clear unit tests and executable specifications.

Executable specifications allow us to verify properties during the simulation. In the example shown in the Listing 7.1 we ensure that there is no two persons in the simulation that have the same id. We use the EventQueue of our simulation to inject this test inside and call it recursively month by month until the simulation is finished. Of course the overall complexity of the simulation will be widened but this ensures that our code respects our specifications.

Beside this specification some *unit tests* are also performed. As explained during the Section 6 we have redefined some operators according to their types. In order to validate the new operators, tests have been performed on each of them. For example, the Boolean type has been redefined into SimuBoolean and a corresponding test has been written into the SimuBooleanTests file. The Listing 7.2 shows one boolean test. According to the *onChange* method, user can define some actions that will be triggered when the value of the SimuBoolean variable is changed. The test simply relies on an external variable that will be set to true whenever the SimuBoolean variable is changed (see source code line 6 in the Listing 7.2).

Those two kinds of tests (unit and executable specification) have ensured us that new operators are properly functional and at any time during the

```
1  import org.scalatest._
2  import org.scalatest.FunSuite
3
4  class PersonTests extends FunSuite with Matchers {
5
6    test("No two people have the same id") {
7      def isDistinct(list: List[Person]): Boolean = {
8        def isContainedIn(p: Person, list: List[Person]):
         Boolean = {
9          list.exists(_.id == p.id)
10       }
11       if (list.isEmpty || list.tail.isEmpty)
12         true
13       else if (isContainedIn(list.head, list.tail))
14         false
15       else
16         isDistinct(list.tail)
17     }
18     def loop(): Unit = waitDelay(1) {
19       assert(isDistinct(population))
20       loop
21     }
22
23     SimModel generatePop (100)
24     loop()
25     SimModel simulate 100.years
26   }
27 }
```

Listing 7.1: Example of one unit test of the class Person

```
1  test("Creation of SimuBoolean & onChange method: init true") {
2    var testSwitch = false
3    var boolTest2: SimuBoolean = true
4
5    /* Adding onChange to switch the testSwitch value */
6    boolTest2.onChange(testSwitch = true)
7
8    /* Check whether the value are correctly set to true at
     initialization */
9    assert(boolTest2.value == true)
10
11   boolTest2 := false
12
13   /* Check whether the value are correctly set */
14   assert(boolTest2.value == false)
15
16   /* Checking if testSwitch has correctly changed */
17   assert(testSwitch == true)
18 }
```

Listing 7.2: Example of one unit test of the class Person

simulation there are no broken specification.

7.1.2 Comparison with Socsim

The second validation objective is to provide results that are similar to Socsim and based on the same configuration parameters and rates. In this section, we will set simulation scenarios and we will compare the results between the two programs.

Test block 1: Validating the Model To validate our model, we must ensure that at least each type of event is planned and executed correctly according to the rate tables and user-specified parameters. The test block number one consists of gradually planning one type of event at a time and plotting the graphs corresponding to this type. To keep the result of MiSca close to Socsim, we use the same pseudo random function as described in the randomness generation 3.1.1. The table undermentioned regroups all the tests that are performed to validate our model.

Num	Test type	Description
1	Death Event & Birth Event	In this first test, only death and birth events will be planned. The other events will be deactivated.
2	Marriage Event& Divorce Event	The second test enables marriage and divorce events.
3	Group Transition Event	Finally group transition will be possible according to specified rates.

The results of this first block test are presented in the result section 7.2.1 and a conclusion about them is done in the Analysis Section 7.3.

Test block 2: Validating the new scheduling process As explained earlier, Socsim plans one event at a time for each person, this can lead to *dormant* person. To compare and validate our new scheduling function, we had to implemented firstly the Socsim scheduling process in order to compare and give similar results. In this test, we will compare the births and the deaths for Socsim and MiSca. Results of this block test 2 are presented in Section 7.2.2.

7.2 Results

This section regroups the raw results obtained based on the methodology described above.

7.2.1 Test block 1: Validating model

Death & Birth Event - Small sized population in 100 years

The input configurations in the **.sup** file of socsim are shown in the following table :

Name	Value	Description
Segment	1	Only one segment will be used
input_file	Appendix 10.1.1	This regroups the initial population that is attached in the appendix
output_file	Appendix 10.1.3	This content of the output_file is joined in the appendix
duration	1200	simulation will last 1200 months
hetfert	0	heterogeneous fertility multiplier is not set to each female

We modified Socsim to use only the birth and death events. Other events such as marriage, divorce and transition are not displayed on those results

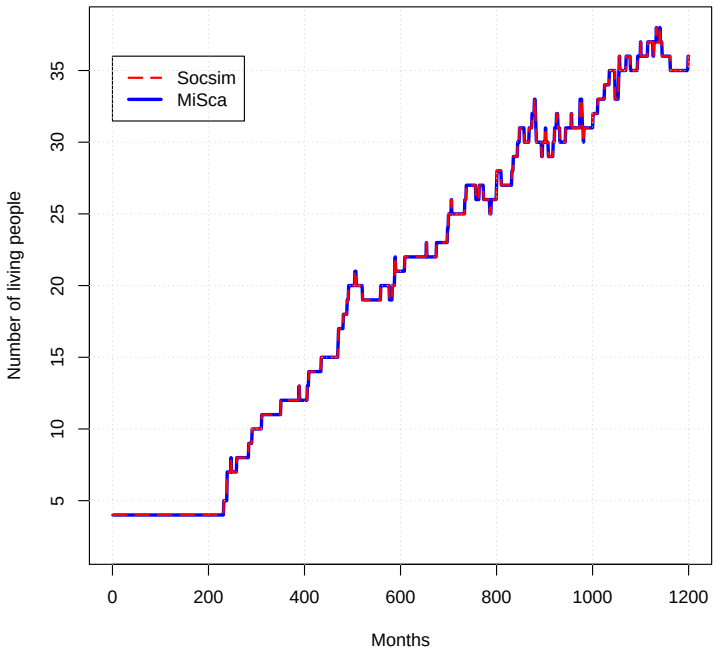


Figure 7.1: Population evolution comparison

since the *rate file*, provided in the Appendix 10.2, set the same rates for every persons. The significance of these tests is to depict how Socsim's and MiSca 's population evolve through ages.

The first graph presented in the Figure 7.1, shows a whole year of population evolution. This population starts from 4 people and grows to 36 alive people for a total of 74 people at the end. As the results demonstrate, the two curves follow exactly the same trend. In order to achieve this, we had to implement the same scheduling event and pseudo-random generation function as Socsim. Earlier tests showed how those two functions can affect the results, those ones are presented in the Threats to validity Section 7.4.

The Figure 7.2 confirms our results with another perspective: the gender and alive distribution follow the same ratio. The picture 7.3 shows a comparison between MiSca and Socsim for a death and birth perspective. As illustrated the birth and mortality rates are exactly the same, every birth and death are displayed by month and follow the same pattern.

Since the simulation starts from a very small population and lasts only one year, this first test aims to validate our model based on birth and death events. The second test increments the population size to three thousand and lasts for two and a half generations.

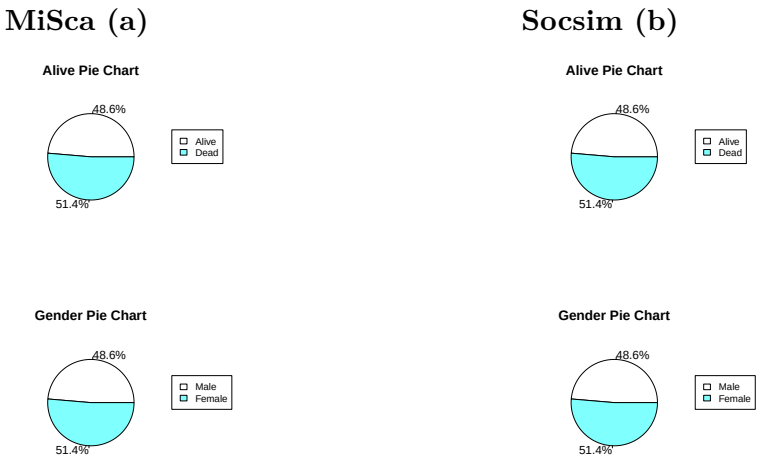


Figure 7.2: Alive and Gender Comparison for a small population of 4 people through a whole year

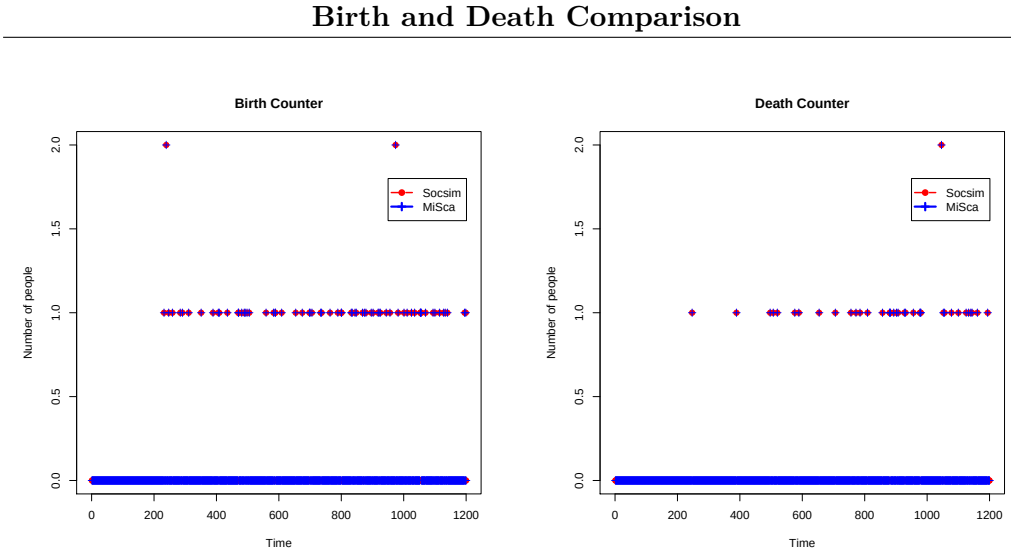


Figure 7.3: Birth and Death Comparison for a small population of 4 people through a whole year

Medium sized population in 250 years

The input configurations in the **.sup** file of socsim were:

Name	Value	Description
Segment	1	Only one segment will be used
input_file	3014 people	This file regroups the initial population where each birth age are set in increasing order from 601 to 1200, other parameters are set to 0
output_file	Appendix 10.1.4	This content of the output_file is joined in the appendix
duration	3000	simulation will last 250 years and will start at month 1200 (according to the last person's birth date in the input file).
hetfert	0	heterogeneous fertility multiplier is not set to each female

In this second test, as shown in the left graph on Figure 7.4, the results appear to be different. The curve does not follow exactly the same trend anymore. This comes from a small difference in the Socsim and MiScaimplementation. This variance originates from the event extract method performed by the two programs. In Socsim, the current month contains the list of events that must be performed this month. However this list is ordered by the smallest person's id whereas MiSca ordered this list by the most recent time. Nevertheless we handle to sort by ids our current event list of the current month and the result is shown in the right figure. This second result illustrates that ordering by id leads increasingly to the same curve. It is important to realize that in this test, even the smallest change in the ordering of events (even whether they are executed on the same month) can lead to such differences in the results. The left figure shows a difference of more than 3000 people at the end of the simulation whereas the one at right-hand side shows only a difference of less than 500 persons.

The Figure 7.5 shows that the ratios are maintained between Male and Female. This results directly from the constant `propMales` which is set by default to 0.5112. But a small difference is noticed between birth and death which suggests that sometimes during the simulation randomized events are not chosen in the same way. We will analyze this difference in the Analysis Section 7.3 with the support of the two graphs regrouped in the Figure 7.6.

Marriage & Divorce Event In this section, marriages were activated. The results are presented on the Figure 7.7. The results are identical to Socsim since they rely on the same implementation.

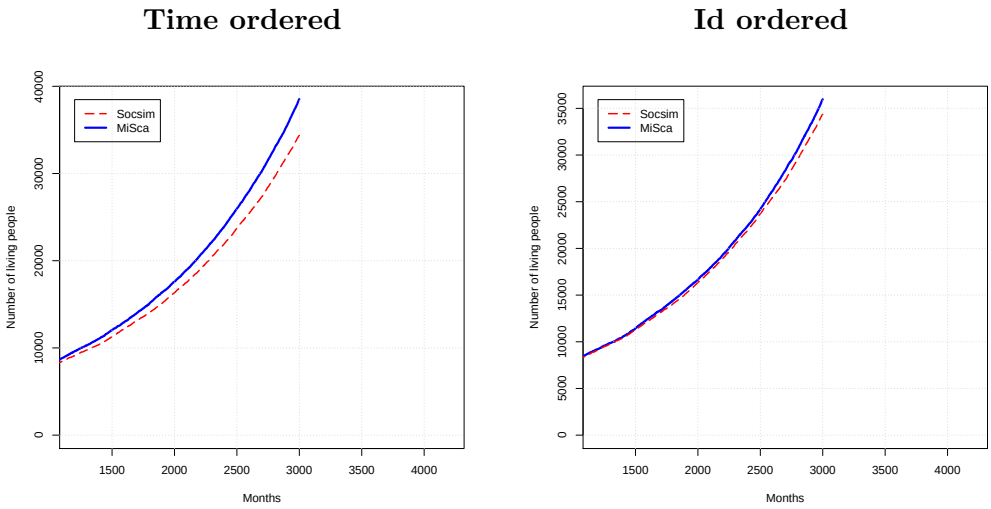


Figure 7.4: Dequeuing method comparison

Socsim

Group (From-To)	Number of people
(1->2)	29
(1->3)	8
(1->4)	15

MiSca

Group (From-To)	Number of people
(1->2)	29
(1->3)	8
(1->4)	15

Table 7.1: Transition Event - Regroups by age people that are still single

Group and Transition Event The transition *rateFile* for this test is shown in the Appendix 10.1.5. Again to keep the simulation simple, we use one segment with all the same mortality and fertility rates for all gender and status. The groups will be splitted by age range, e.g., the first group concerns about 0 to 9 years old, the second group 10 to 19, etc. Results of this test classify single people according to their age into their respective group. Rates are only set from the group 1 to all the others(uni-directional), meaning that once the person goes into one group he cannot leave this last one. The purpose is to ensure that nobody leaves the group once entered. The population file regroups people with increasing birth date in order to separate them in the different groups. The results of this test are shown in the Table 7.1.

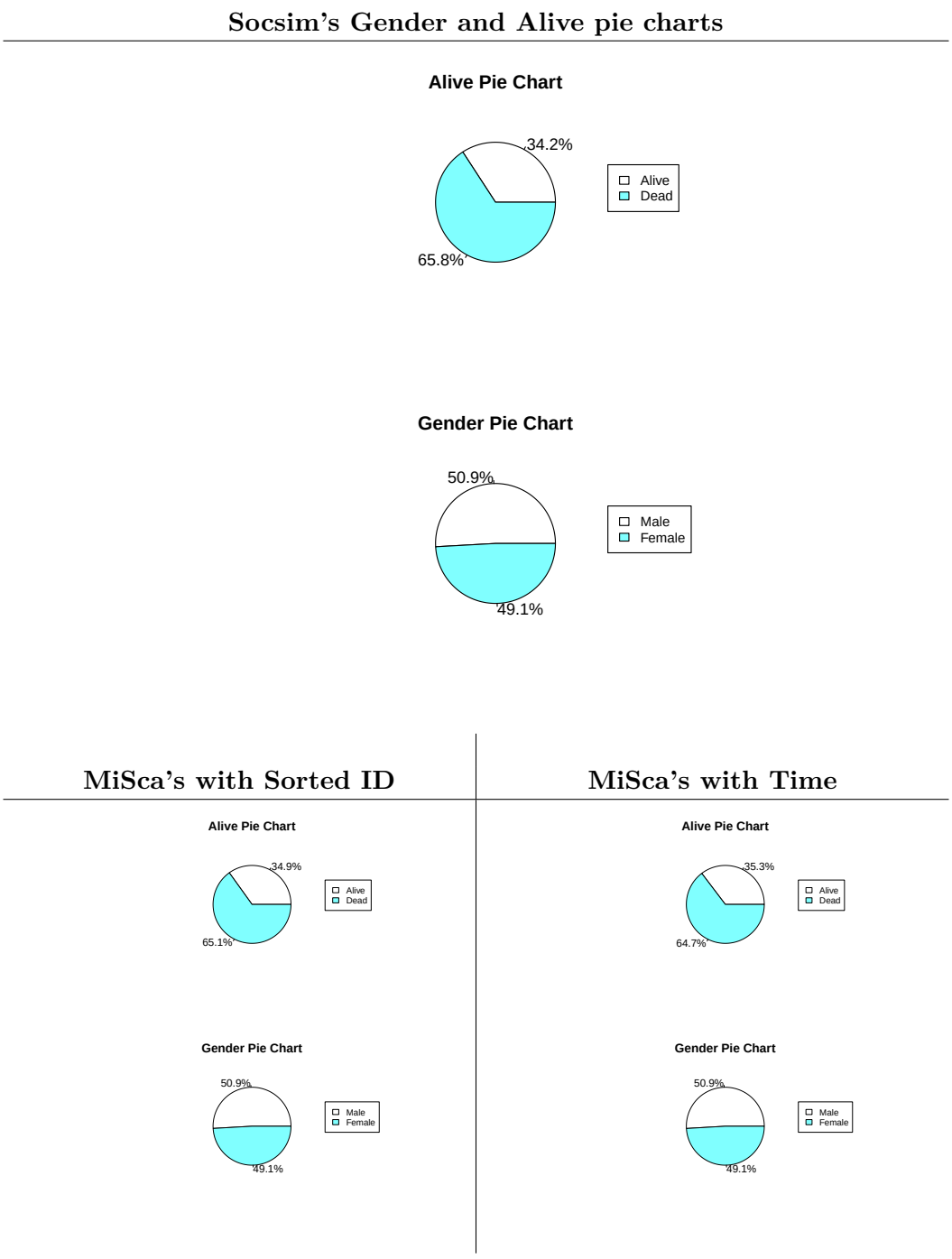


Figure 7.5: Gender and Alive Comparison with the Event Queue ordered by person's ID and time

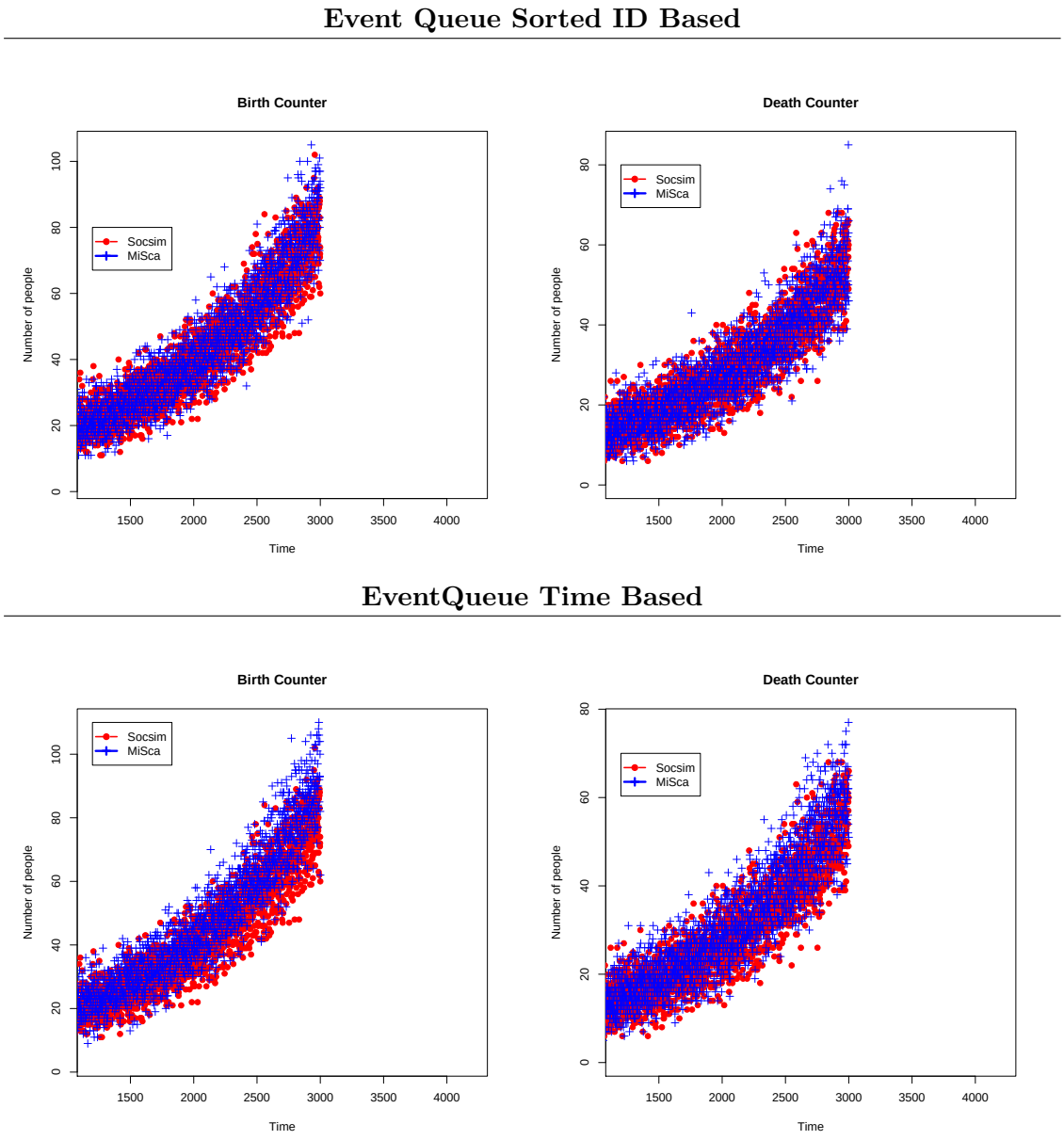


Figure 7.6: Birth and Death Comparison with the Event Queue ordered by person's ID

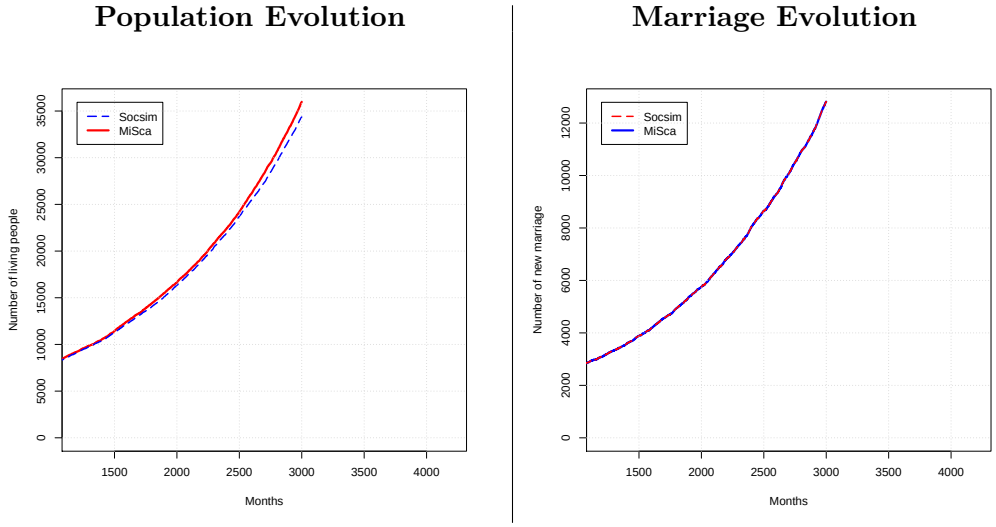


Figure 7.7: Marriage & Divorce Event Comparison

7.2.2 Test block 2: Validating the new scheduling process

MiSca, as explain in the Solution 6, relies on multiple events scheduled per person. We will present here results of 3 sub-tests. Those tests take the same input population: a small group of 4 people for which only their segment duration are changed. The duration is set to 250, 500 and 1000 years. Again the same fertility and mortality table are used 10.2.

The Figure 7.8 regroups the results for the 3 durations. As you can see the first result (on left-hand) seems to be odd since the two curves does not follow each other. However it seems that besides their different values they follow the same trend, like a sinusoidal wave. The MiSca's curve is an upscaled version of the Socsim's one: on the first steepening of the curve, they follow the same trend. As MiSca can plan more events, this first gradient is a direct result of the new scheduling process. You can use the next figure 7.9, representing the birth and death counter, to understand that MiSca plans more events than Socsim. Back to our first graph, on the next slope, results show for both an equivalent declination at the 500th months proportional to the number of people at this time. The rest of the curve follows the same waving movement. It seems interesting to go through a longer period to see how those curves stabilize themselves. The next picture, at the center, shows the same scenario but the simulation lasts twice longer. In this case, curves tend to stabilize themselves during the first three decades but the MiSca curve grows exponentially after this time. The Socsim one grows too but not as much. The last picture, on right-hand side, shows that same pattern happening to Socsim but with a lower scale. Based on those results a conclusion will be made in the Analysis Section 7.3.

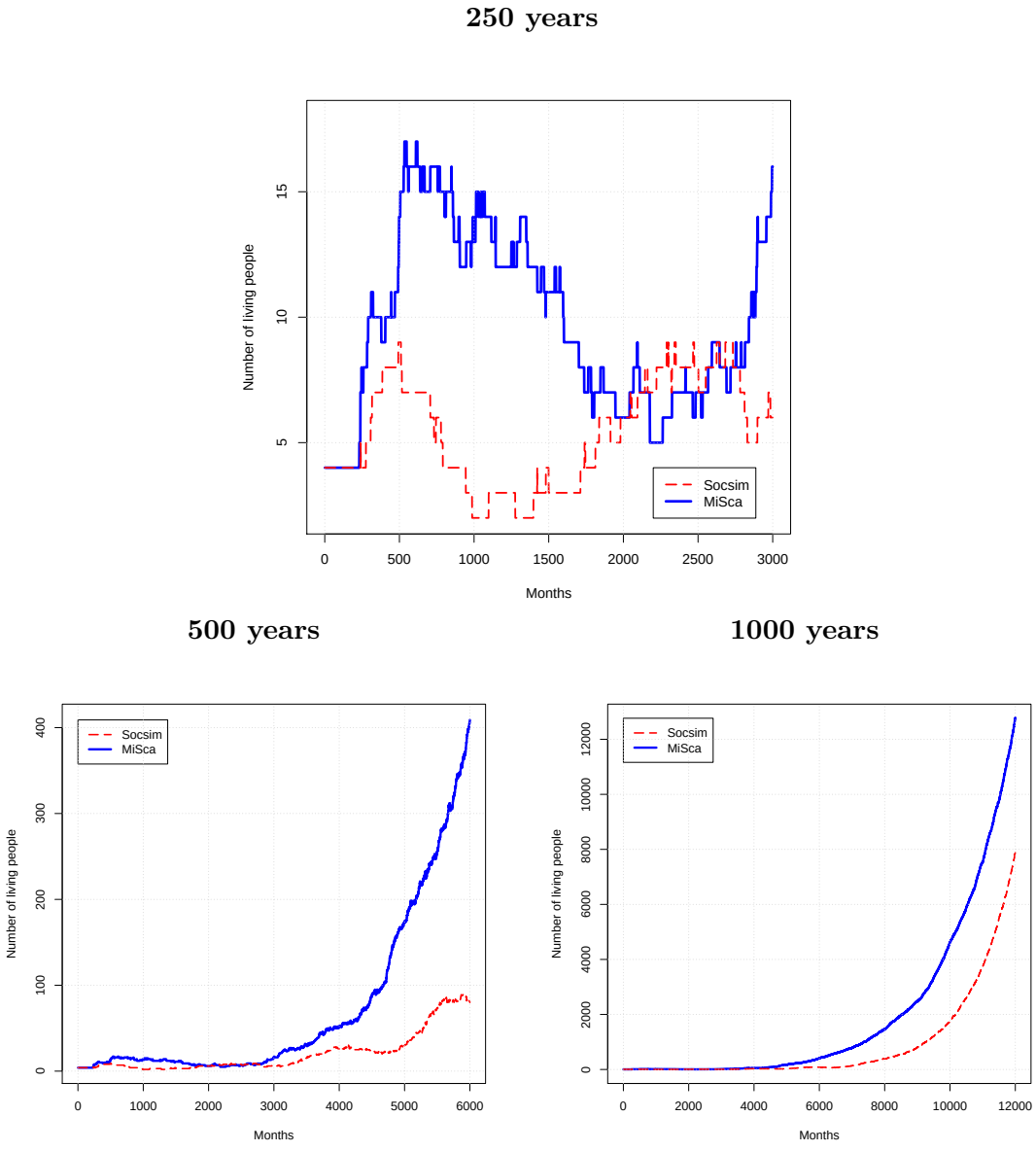


Figure 7.8: Population evolution of Socsim and MiSca, with the last one using new scheduling process

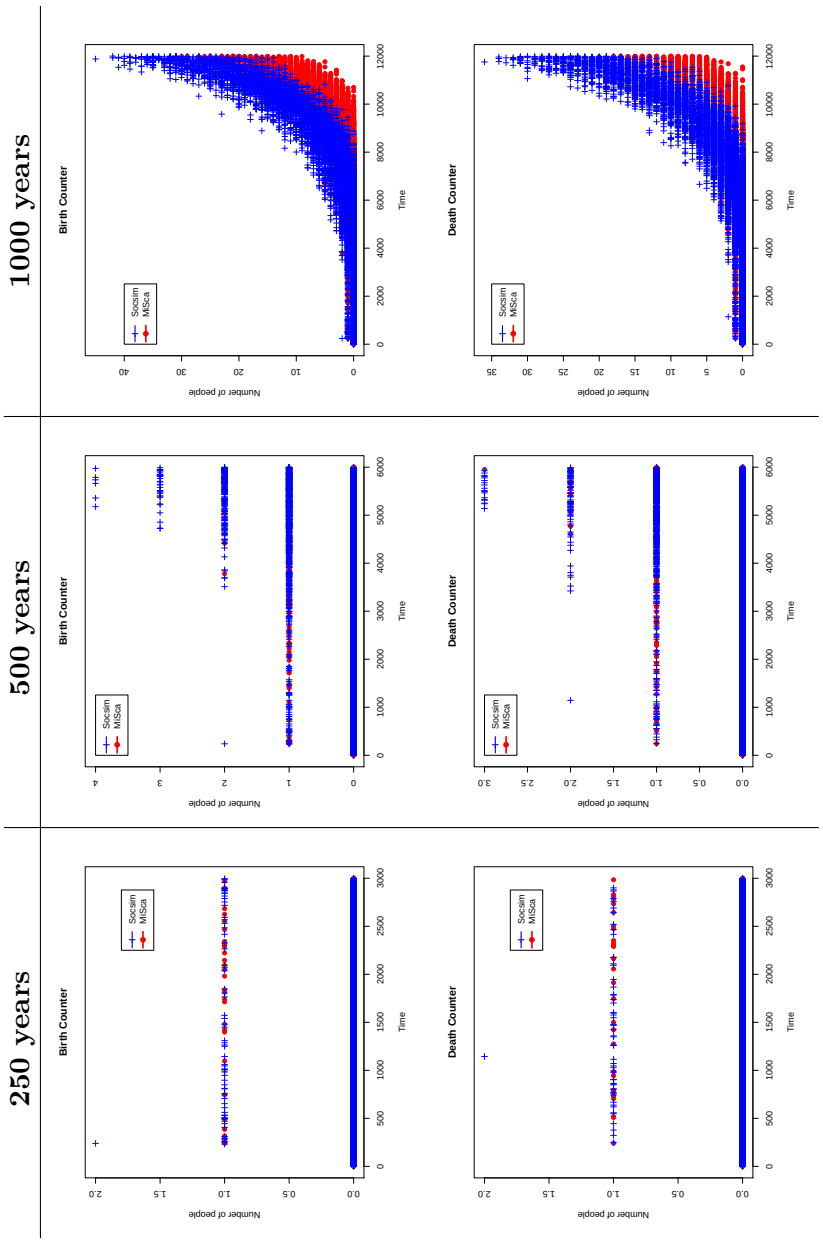


Figure 7.9: Population evolution of Socsim and MiSca, with the last one using new scheduling process

Based on these Testing blocks, analysis will be performed in the next section 7.3.

7.3 Analysis

This section draws conclusion of the results shown in the previous section.

7.3.1 Test block 1: Analysis

This first test aims to validate our implementation with Socsim. As the first graphs show, results are identical in the short population but small variation appears in the second part. This second simulation runs during 250 years and tends to be different from Socsim at the end. To cope with the Socsim's curve, the first modification was to force the utilization of an ordering of the events that are executed for the current month. Socsim goes through events on a monthly basis, and pops out events of the current month that are associated with the person's id. Since our event queue is implemented as a priority queue, events are classified along with their respective time. (shorter comes first). The solution was to pop out all events of the same time (the current month) and then order them by id. The result shows an amelioration in the curve (see graph 7.4). But this "unnecessary" operation in MiSca causes overhead since each group of events with the same time must be reordered by id. Notice that this reordering is just for the purpose of having the same results as Socsim. User can still have the choice to use this reordering or not whether they want to retrieve results as close to Socsim.

Nevertheless the curve is not yet the same. The reason of this is, as we believe, an inconsistency in the Socsim code during the planning. Recall from earlier the planning in Socsim plans at least one event per person, meaning that at least the death event is scheduled for a person. This death event will be scheduled at the *end of his life* which is supposed to be 120 according the the constant *MAXUYEARS*. The problem arises when the user configures the person *MAXUYEARS* constant to 120 which is the maximal human age but the planning function does not rely on that constant. The value 1200, corresponding to 100 years is hard-coded in the main *while* loop of the planning operation. Due to this problem, even whether user configures a death age limit to 110 or 120, Socsim will not plan event passed 100 years. MiSca uses directly the constant *AgeLimit* to set up the upper bound limit to a person plannification. When we set up the constant to 100 the results were the same as Socsim. To motivate this answer let us take again the birth and death counter graphs at the Figure 7.6. In this graph, the blue crosses are most of the time a little bit above the mean red dotted values. This is a direct consequence of the age limit. Moreover, in the first test, as

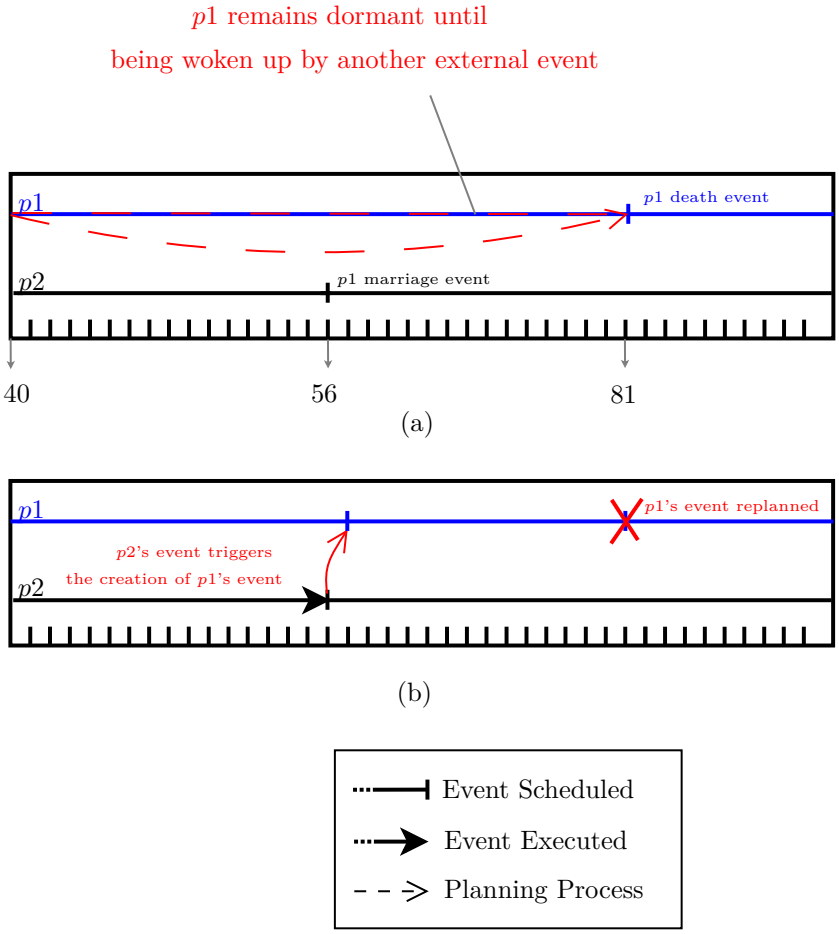


Figure 7.10: Socsim's domant person

the simulation stops at 100 years, the results were the same (identical curves 7.1) because the *age limit* value was not reached yet.

The next incremental tests with the marriages on Section 7.2.1 and transitions on Section 7.1 shows identical results as those events are respectively implemented the same way in MiSca.

7.3.2 Test block 2: Analysis

This test aims to validate our new scheduling function and secondly to illustrate the *dormant* person in Socsim. Recalling from the results, the curves followed the same trend but the MiSca's tended to rise faster. As can be seen in the Figure 7.9 the amount of births and deaths in MiSca are more important that in Socsim. This comes from the difference between one event planned and multiple events planned per person. As previously mentioned

during the Socsim overview, *dormant* person may arise whether the planning function sets the death of a person $p1$ at the beginning of its life. An example is shown in Figure 7.10. In this first scheme (a), the woman $p1$ has planned a death to 81 years. Since only one event is possible for $p1$ she will remain inactive from 41 to 81 years old. If no other people interacts with her during her life, she will remain dormant and consequently she cannot have births, self initiated marriage, etc. The sub-figure (b) shows the only way to wake up the woman. In this case, $p2$ generates a marriage with $p1$ which leads to re-plan an event for the woman to change her marital status. In MiSca such a situation cannot be possible since $p1$ can plan, in the same planning operation, a most a death, a birth, and a marriage (or divorce). This different approach alters directly the results showed in the Test block 2 in Section 7.2.2, since the number of events are more important in MiSca.

7.4 Threats to validity

7.4.1 Previous model used

First solution Our first choice was to design intuitively a microsimulation model which relies on monthly probabilities for the occurence of events. Since fictitious persons in the simulation may or may not trigger some events, randomness must be used and compared to a certain probability. Socsim used rate tables that are difficult to understand at first glance because they rely on rates established by month and defining its own rates can be difficult. The most appropriate probability law seems to be the *geometric distribution*: the probability of the first occurrence of success requires k number of independent trials, each one with a success of probability p. The basic idea is therefore to associate a certain probability p to a certain age range and for each trial (attempt failed) re-roll a virtual die and try again. Each trial failed will increase by one month the event waiting time. The geometric distribution can be viewed as:

$$X \sim Ge(p) \text{ if and only if :}$$

$$p(x) = \begin{cases} p(1-p)^{x-1} & \text{if } x = 1, 2, ... \\ 0 & \text{otherwise} \end{cases} \quad ; \; F(x) = 1 - (1-p)^x \; x = 1, 2, ...$$

This first model was quite easy to understand and to use since to define rate in the configuration one could use rate table such as Socsim but with probabilities instead of rates :

```
death 1 M single
0 1 .0001
20  .005
30 0 .005
```

```
60 0 .009
80 0 .03
100 0 .7
```

The following rate table gives the mortality rates for males who are single. The first row specifies the year of occurrence, e.g., the first row specifies an age range of 0 to 1 month. The second row specifies the months and the third indicates the probability of occurrence. Therefore the table above says: A single male aged 20 or less has a probability of dying sets to 5 %. A man aged between 20 and 30 years old has a probability of dying of 5 %, etc. To give you a comparison Socsim does not rely directly on probabilities but uses rates that cannot be understandable at first glance:

```
death 1 M single
0 1  .460940
20  .0057540
30 0 .008730
60 0 .002600
80 0 .0832630
100 0 .832630
```

The results of this implementation was conclusive but a problem rose during the validation. Since the probability must be provided for filling the mortality, fertility, etc. we cannot easily validate this model because it does not rely on any "true" demographic tables that are surveyed.[1] A second solution was to use rates and convert them into probabilities.

Second solution In order to keep the geometrical law from the first solution, we use translation function to convert Socsim rates (or other microsimulation model into probabilities). Two translation functions were developed and are still usable in MiSca :

```
1   val brassLogit:
2    Double => Double = (x => 1 / (1 + exp(-2 * x)))
3
4   val coaleDemeny:
5    (Double, Int) => Double = (x, y) => (1 - (pow(1 - x, y)))
6
7  /* Those 2 functions can be used in the translate argument */
8   def parseRates(file: String, translate: ((Double, Int) =>
    Double))
```

[1] United Nations Statistics Division: `http://unstats.un.org/` provides mortality and fertility rates into a demographic yearbook.

The function comes directly from the Manuel X[11], where several demographic models are explained. Recall from the Section 2.4 the two functions concerns the *Coale and Demeny regional model* and the *Brass logit system*. Based on this two functions MiSca can use the logit values for the *Brass* general standard life table or the parameters from the Coale and Demeny regional life-table.

Due to lack of comparison with other existing tools that use such demographic models, those conversion functions are until now not validated but are still usable in MiSca.

7.4.2 Pseudo random generation

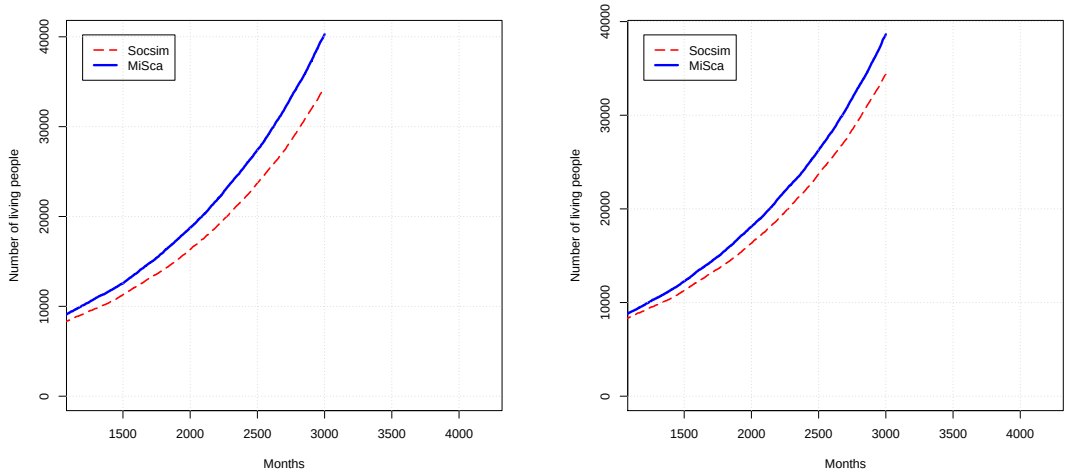
Previously mentioned during the Results section, in order to validate our model with Socsim, our program must follow as close as possible the Socsim's model. This imposes that we use the same random function than generates sequence of number based on a seed number. In this section we will illustrate why we need to implement this random function and why it is important for the results' validation.

The Figure 7.11 shows the comparison of Socsim and MiSca for a simulation of 250 years. Exactly the same parameters and rates were used during this simulation except for the seed number. Socsim random suite were generated with seed sets to 12345 and MiSca with 17845 and 60000. This number is chosen in order to illustrate the difference that can be observed when the two identical simulations are launched with different seed numbers. The last graph in this figure shows the utilization of the scala.util.Random function. The results of the two upper graphs show that the trend of the curve is affected but not dramatically, however the simulation is performed on only two generations.

7.4.3 Time complexity

As the previous results illustrate, several transformation were made to MiSca to validate the results with Socsim such as reordering the Event Queue based on the person's id to retrieve the person in the same order. Moreover to use the same pseudo random, a special rounding (half up with 6 decimals maximum) was use in order to retrieve a similar seeds. Along with this transformation MiSca objective was to provide a better expressiveness. This gain is unfortunately at the expense of the program's speed. Since all the observable attributes or custom events must be inserted in the Event Queue, this last one must deals with bigger size. Moreover, depending of the frequency of the events (e.g., collecting data each month will introduce at least one event per month to gathering those data) the queue can quickly reach a huge size. This results in executing each month events and hence give the same performance as an *Activity based simulation* where the simulation go

Seed number variation



17845 60000

Scala random function

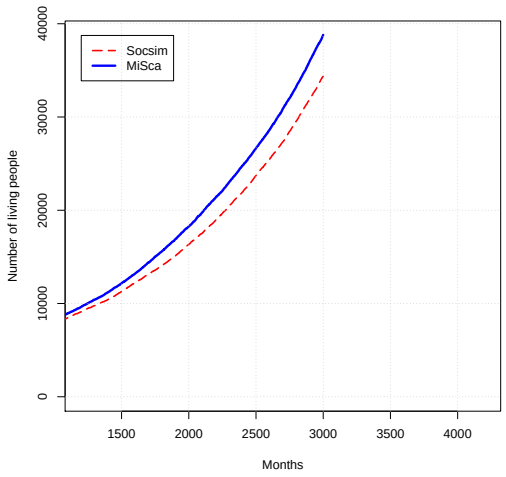


Figure 7.11: Comparison between Socsim and MiSca with different random functions

through monthly. Several improvements were nevertheless made to retrieve and store quickly rates of rate tables. The usage of Scala Map ensures that rates are stored and get at a constant time. Whether the simulation last longer than expected, the usage of section can be applied to split up the simulation and store intermediate results in output files.

7.5 Conclusion from the analysis

Results of the two tests were conclusive, the first testing block shows very high similarity with Socsim with all the different types of events, and even through long simulations. The analysis has revealed an uncertainty about the Socsim age limit variable which leads to different results at the second part of this test. Nevertheless, with same attributes and initial configuration MiSca gives results that are similar to Socsim.

The second test validates our new scheduling function as described in the Solution Section 6, this function reveals interesting behavior as it follows the same trend as Socsim but with a higher growth.

Some other tests were made to demonstrate the variation of results based on the choice of a pseudo random number that is used to generate the suite of numbers. Based on this variation, it is important to notice that such a pseudo random function is mandatory if the user wants to reproduce precisely the same microsimulation conditions. Without this function, variation of the number of people is proved but the overall proportion and the general trend of the curve stays the same.

Based on this short conclusion we will now give our final results and resume how we handle the problem exposed.

Chapter 8

Future work

As the validation process reveals, there is some room for improvements. This section aims to present some suggestions for improvement.

Improving time complexity - As a time complexity problem was observed, some work has to be done for decreasing the execution time. One suggestion would be to analyse if data structures, such as the priority queue (used for queuing events) or the hashmaps (used to contain rates), are well-suited. However it is still difficult to get better results than those obtained with C programming language.

Supporting the parallelization - Small improvements can be made during the planning operation. Since at each planning, every type of events must be evaluated independently in order to execute only the shortest one (if we use the Socsim scheduling) or to plan one for each type (MiSca's default implementation). Enhancing execution of each type of planning in a thread may result in a complexity improvement. Other kinds of parallelization are not possible in the microsimulation since events are competing with each other. Moreover sections cannot be put in threads since the input population of the next section depends on the output of the previous one.

GUI continuation - The current Graphical User Interface of MiSca relies on a mini web-server that is hosted on the user computer. Currently only the basic charts are modeled with the help of d3js but many other ones can be added very quickly since MiSca contains functionalities to output formatted JSON files with user's specified data.

DSL improvements - Even if the current DSL is already expressive and communicative, some improvements are always possible. For example, a DSL related to GUI should be designed when this one will be more accomplished.

Chapter 9

Conclusion

During this master's thesis, we developed a new microsimulation software that aims to reuse the basic concepts of other well-known tools such as Socsim, while incorporating new ones. The program had to be as flexible and expressive as possible in order to facilitate the creation of specific demographic scenarios. The advantages of the Scala programming language were exploited to build a powerful and expressive DSL. This one helps the user to manipulate easily the program components and attributes in order to customize and create new functionalities. An example of a HIV-DSL were used to illustrate the expressiveness of the tool.

In order to provide a decent software, many existing tools and literature about microsimulation were studied. The conception of the program went through many different core implementations to achieve a coherent and reliable structure. The program was driven by simplicity (in order to provide a clear and understandable source code) and reliability through results comparison against Socsim. The validation with unit tests and executable specifications has reinforced the consistency of the tool.

The flexibility is obtained by facilities such as observers allowing to have control on almost all the entities of MiSca. However these high level manipulations tend to rise the overall execution time of the software.

Nevertheless some features were developed to counteract this time complexity problem. For example sections allow to cut the execution into smaller simulations in order to obtain intermediate results.

As a conclusion we observe that major objectives are achieved. Especially, the validation process demonstrates that MiSca achieves similar results to those obtained by Socsim. Moreover, through examples, we also proved that the designed DSL allows to express scenarios in a communicative way.

Chapter 10

Appendix

10.1 Input and Output File of the Scheduling validation

10.1.1 Input_file

```
1 1 1 0 0 0 0 0 0 0 0 0 0
2 1 1 0 0 0 0 0 0 0 0 0 0
3 1 1 0 0 0 0 0 0 0 0 0 0
4 1 1 0 0 0 0 0 0 0 0 0 0
```

10.1.2 UML events

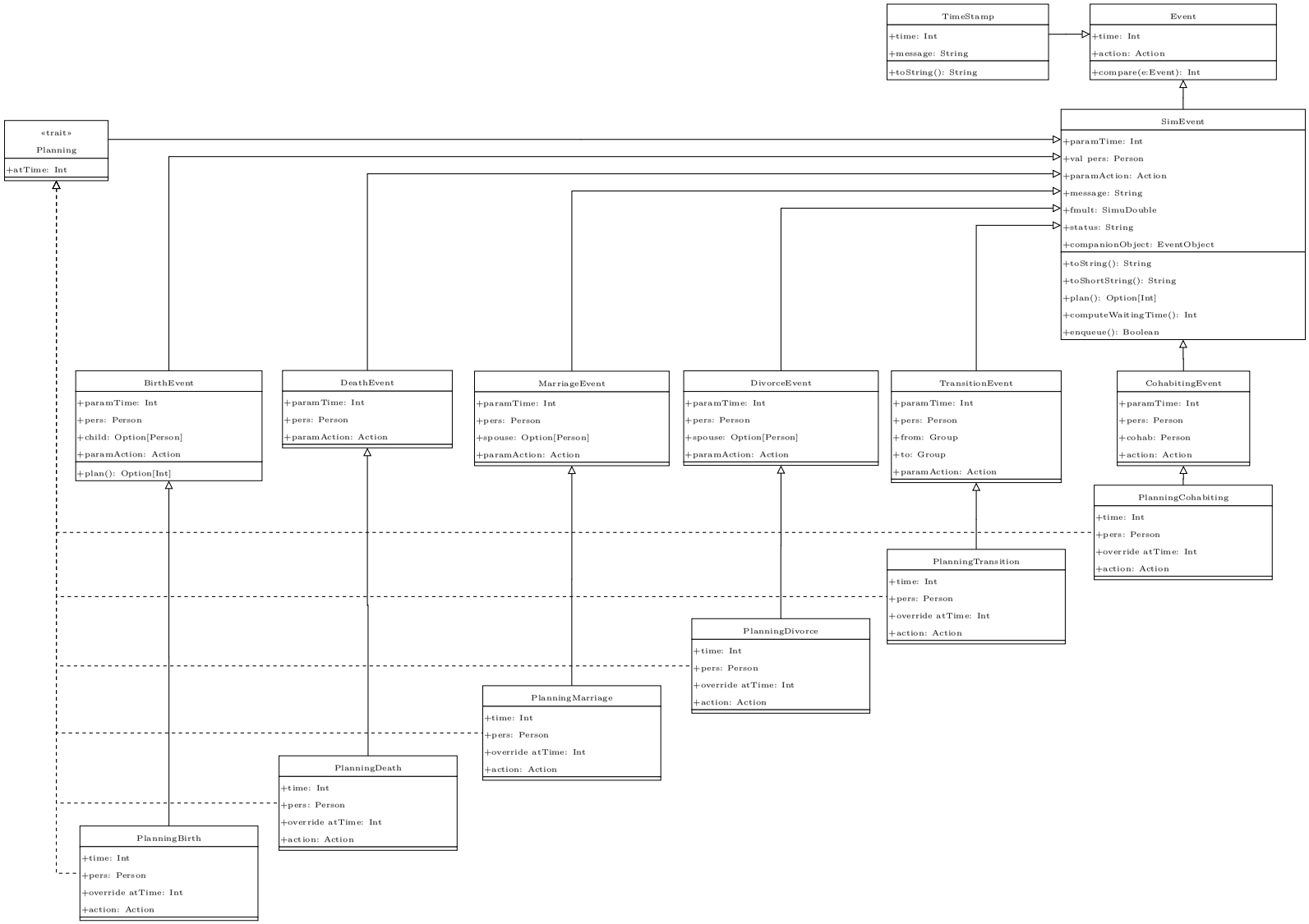


Figure 10.1: MiSca events

10.1.3 Output_file 100 years

```
Socsim Version: STANDARD-UNENHANCED-VERSION

Segment NO: 1 of 1 set to run with following macro options
Duration: 1200

 Simulation Complete
writing population...marriages..total size of pop 74

living size of pop 36

Population Pyramid at the end of Segment 1  Month: 1201
             Group: 1  Total Population: 36
---------------MALE--------------------|--------------FEMALE---------------
                                       |
                                       |
                                       |
                                       |
                                       |
                                       |
                                       |FFFFFFFFFF
                 MMMMMMMMMMMMMMMMMMMM|
                                       |FFFFFFFFFF
                                       |FFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
                             MMMMMMMMMM|
                             MMMMMMMMMM|FFFFFFFFFF
                 MMMMMMMMMMMMMMMMMMMM|FFFFFFFFFF
                             MMMMMMMMMM|FFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
                 MMMMMMMMMMMMMMMMMMMM|
MMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMM|
                 MMMMMMMMMMMMMMMMMMMM|FFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
                                       |FFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
                 MMMMMMMMMMMMMMMMMMMM|FFFFFFFFFF
                             mmmmmmmmmm|ffffffffff
   ------+---------+---------+---------+---------+---------+---------+------
         15        10         5         0         5        10        15
```

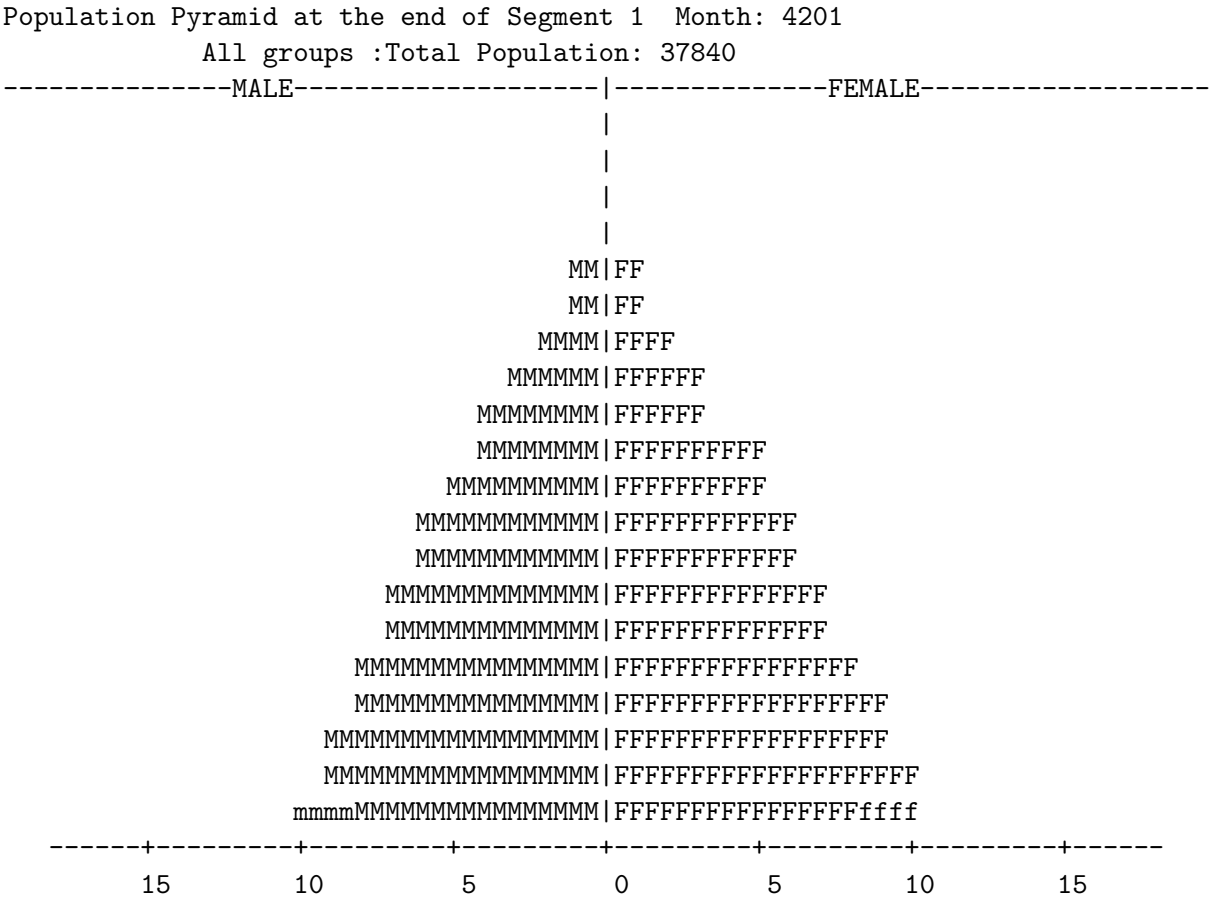
10.1.4 Output_file 250 years

```
Socsim Version: STANDARD-UNENHANCED-VERSION

Segment NO: 1 of 1 set to run with following macro options
Duration: 3000

 Simulation Complete
writing population...marriages..total size of pop 106565

living size of pop 37840
```



10.1.5 Transition Rates

```
1 1 1 0 0 0 0 0 0 0 0 0 0
2 1 1 0 0 0 0 0 0 0 0 0 0
3 1 1 0 0 0 0 0 0 0 0 0 0
4 1 1 0 0 0 0 0 0 0 0 0 0

% Male Group Transition

transit 1 M single 2
20 0 0
30 0 .99
100 0 0

transit 1 M single 3
30 0 0
40 0 .99
100 0 0

transit 1 M single 4
40 0 0
50 0 .99
100 0 0

transit 1 M single 5
50 0 0
60 0 .99
100 0 0

transit 1 M single 6
60 0 0
70 0 .99
100 0 0

transit 1 M single 7
70 0 0
80 0 .99
100 0 0

transit 1 M single 8
80 0 0
90 0 .99
100 0 0

transit 1 M single 9
```

```
90   0   0.000
100  0   .999

% Female Group Transition

transit 1 F single 2
20 0 0
30 0 .99
100 0 0

transit 1 F single 3
30 0 0
40 0 .99
100 0 0

transit 1 F single 4
40 0 0
50 0 .99
100 0 0

transit 1 F single 5
50 0 0
60 0 .99
100 0 0

transit 1 F single 6
60 0 0
70 0 .99
100 0 0

transit 1 F single 7
70 0 0
80 0 .99
100 0 0

transit 1 F single 8
80 0 0
90 0 .99
100 0 0

transit 1 F single 9
90   0   0.000
100  0   .999
```

10.2 Rate Table Test Blocks

Mortality Table

Age	Months	Rate
0	1	0.088135
1	0	0.0083525
5	0	0.0014636
10	0	0.00034839
15	0	0.000191579
20	0	0.000310663
25	0	0.000453001
30	0	0.000460706
35	0	0.000513193
40	0	0.000631281
45	0	0.000810408
50	0	0.00107293
55	0	0.00147561
60	0	0.00207753
65	0	0.00300088
70	0	0.0045236
75	0	0.00697976
80	0	0.0108844
85	0	0.0163323
90	0	0.0251368
95	0	0.0385987
100	0	0.0590448

Fertility Table

Age	Months	Rate
15	0	0.0
16	0	0.001667
17	0	0.001667
18	0	0.003333
19	0	0.006667
20	0	0.013333
21	0	0.018833
22	0	0.018833
23	0	0.018833
24	0	0.018833
25	0	0.018833
26	0	0.018333
27	0	0.0175
28	0	0.015833
29	0	0.013333
30	0	0.011667
31	0	0.01
32	0	0.008917
33	0	0.00875
34	0	0.008667
35	0	0.008667
36	0	0.008667
37	0	0.008333
38	0	0.008167
39	0	0.008
40	0	0.007667
41	0	0.007083
42	0	0.00625
43	0	0.005
44	0	0.004167
45	0	0.003333
100	0	0.000

Table 10.1: Tables used in the validation part, for all the status and gender

10.3 Running example & GUIs

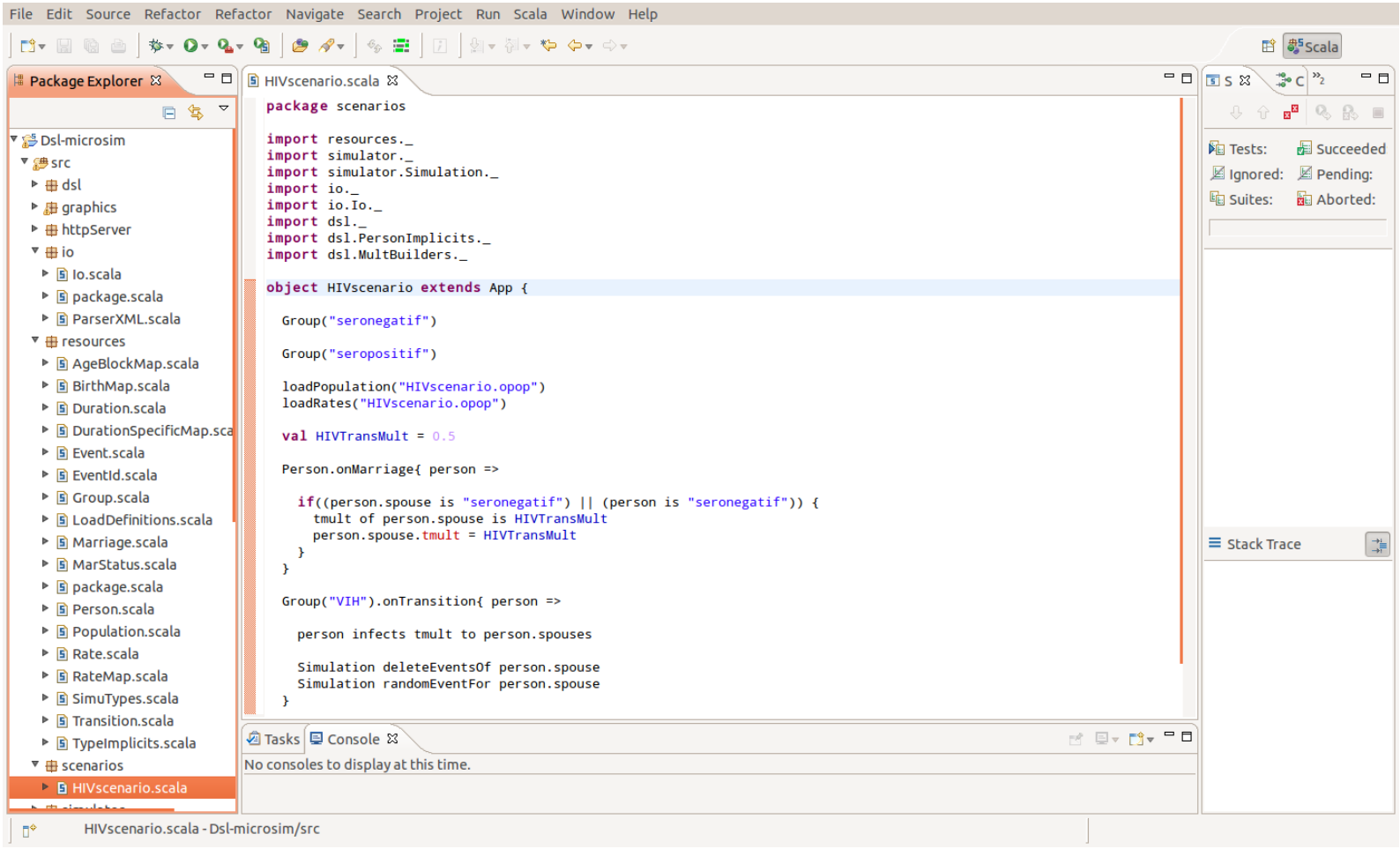


Figure 10.2: Running example in Eclipse environment

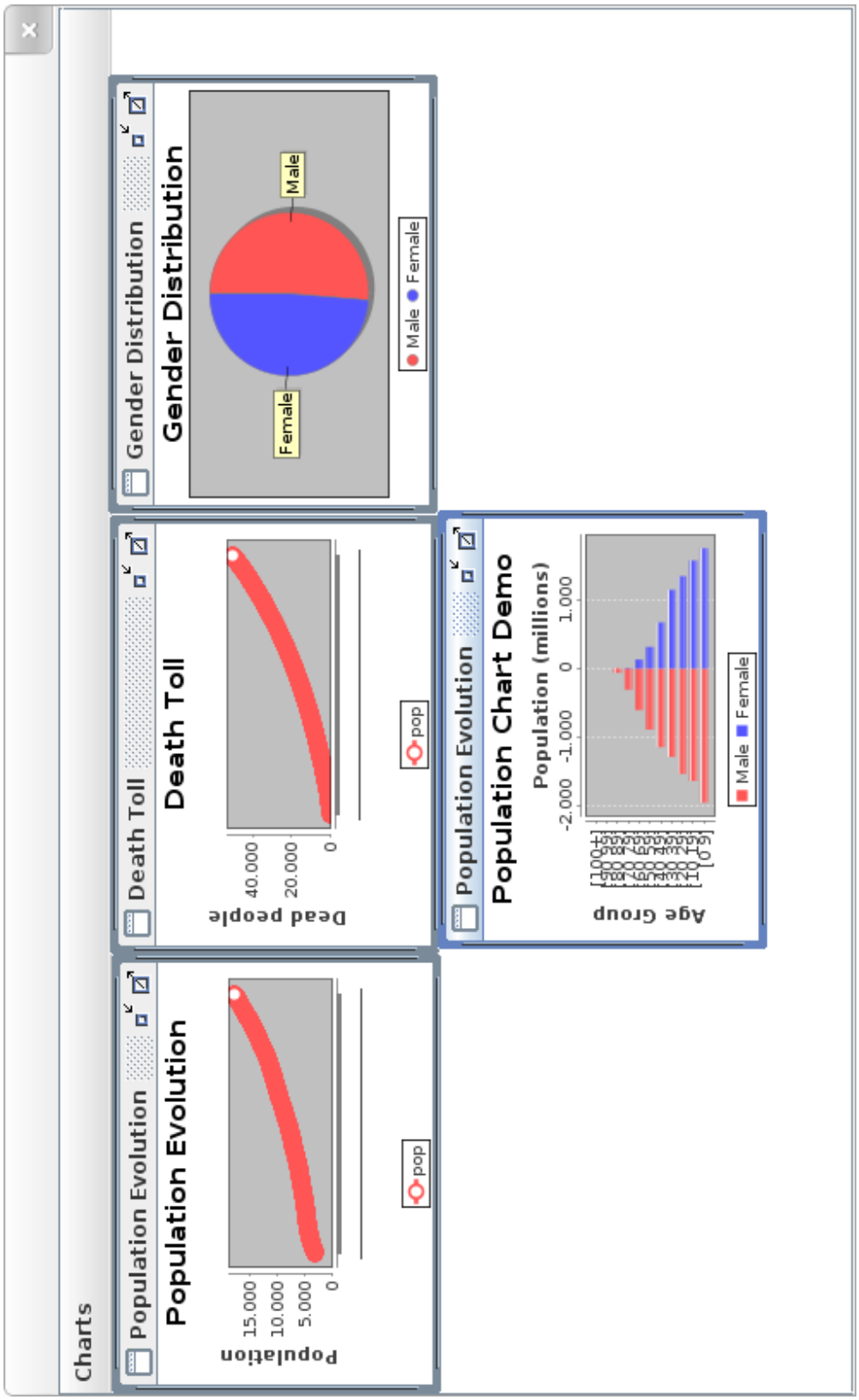


Figure 10.3: Illustration of the first GUI

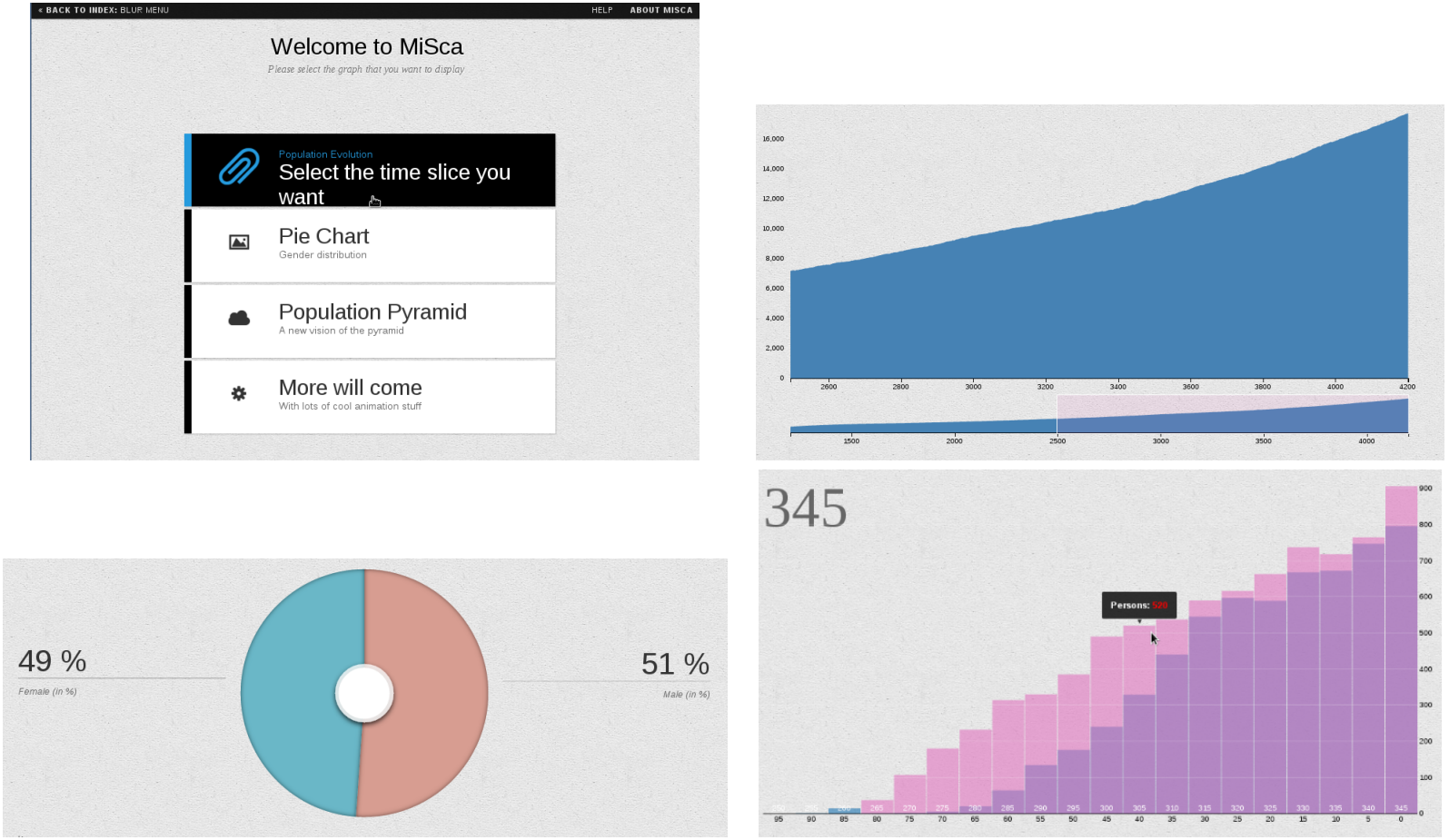


Figure 10.4: Illustration of the second GUI

Bibliography

[1] I. M. Association. `http://www.microsimulation.org/`, July 2014.

[2] E. Gamma, R. Helm, R. Johnson, and J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1995.

[3] D. Ghosh. *DSLs in action*. Manning Publications Co., 2010.

[4] E. V. Imhoff and W. Post. Méthodes de micro-simulation pour des projections de population. *Population*, page 4:889–932, 1997.

[5] C. Mason. Socsim oversimplified. *demog.berkeley.edu*, august 2013.

[6] B. Masquelier. *Estimation de la mortalité adulte en Afrique subsaharienne à partir de la survie des proches. Apports de la microsimulation*. PhD thesis, UCL, centre de recherche en démographie et société, november 2010.

[7] B. Masquelier. Quelques notes sur la microsimulation en démographie, mai 2013.

[8] N. Matloff. Introduction to discrete-event simulation and the simpy language. *Davis, CA. Dept of Computer Science. University of California at Davis. Retrieved on August*, 2:2009, 2008.

[9] J. A. Miller, J. Han, and M. Hybinette. Using domain specific language for modeling and simulation: Scalation as a case study. In *Simulation Conference (WSC), Proceedings of the 2010 Winter*, pages 741–752. IEEE, 2010.

[10] C. Murray, B. Ferguson, A. Lopez, M. Guillot, J. Salomon, and O. Ahmad. Modified logit life table system: Principles, empirical validation and application. *World Health Organization, GPE Discussion Paper No.39*.

[11] U. Nations. *Manual X: Indirect Techniques for Demographic Estimation*. United Nations, 1983.

[12] M. Odersky, L. Spoon, and B. Venners. *Programming in Scala*. Artima, second edition edition, 2010.

[13] R. M. i. D. Princeton Univeristy. The lee-carter model, Spring 2009.

[14] S. Ruggles. Confessions of a microsimulator: problems in modeling the demography of kinship. *Historical Methods: A Journal of Quantitative and Interdisciplinary History*, 26(4):161–169, 1993.

[15] P. Schaus. Languages and translators course, 2012-2013.

[16] SOCSIM. Microsimulation with socsim. `http://lab.demog.berkeley.edu/socsim/`, July 2014.

[17] M. Spielauer. The lifepaths microsimulation model: An overview. `http://www.statcan.gc.ca/microsimulation/pdf/lifepaths-overview-vuedensemble-eng.pdf`, September 2013. Statistics Canada - Modelling Division.

[18] B. Venners, G. Berger, and C. C. Seng. Scalatest. `http://www.scalatest.org/`, 2007.

[19] Wikipedia. Domain-specific language. `http://en.wikipedia.org/wiki/Domain-specific_language`, July 2014. Online; accessed 25-July-2014.

[20] Wikipedia. Microsimulation. `http://en.wikipedia.org/wiki/Microsimulation`, July 2014. Online; accessed 16-July-2014.

[21] U. Wilensku. Center for connected learning and computer-based modeling. `https://ccl.northwestern.edu/netlogo/`, 1999. Northwestern University, Evanston, IL.

[22] Z. Zhao. Computer microsimulation and historical study of social structure: A comparative review of socsim and camsim. *Revista de la Asociacion de Demografia Historica*, 24(2):59–88, 2006.